

基于非线性规划与 PSO 算法的烟幕干扰弹投放策略研究

摘 要

本文主要研究了无人机完成烟幕干扰弹的投放策略优化问题。根据导弹、无人机与烟幕云团的运动学与消散特性,利用**非线性规划模型**与空间几何判定方法,结合改进**粒子群优化(PSO)算法**、**时间步进法**与**二分法**,对多场景下无人机的飞行参数与干扰弹投放策略进行优化求解,并系统分析了不同协同模式下的遮蔽效能。

针对问题一,首先明确完全遮蔽的标准,根据给定的固定参数条件,通过建立导弹、无人机、干扰弹及云团的运动学模型,结合基于圆柱体轮廓**关键点采样**的几何遮蔽判定方法,并采用**时间步进法**与**二分法**进行高精度数值计算,求解得到烟幕云团对导弹 M1 的有效遮蔽时长为 **1.3916 秒**。

针对问题二,以最大化单机单弹遮蔽时长为目标,根据无人机、干扰弹的运动约束与云团遮蔽机制,建立**四维非线性规划模型**,利用改进**粒子群优化(PSO)算法**进行全局寻优,求解得到 FY1 的最优投放策略为:飞行方向角 6.951° 、速度 139.55 m/s 、投放时间 0.0000s 、起爆延时 0.7405s ,最大遮蔽时长为 **4.588 秒**。

针对问题三,在 FY1 连续投放 3 枚干扰弹的场景下,考虑多弹协同遮蔽与时间-空间衔接约束,建立八维非线性规划模型,引入多弹协同判定逻辑,采用高维改进 **PSO 算法**进行求解,得到最优策略为三弹投放时间依次为 0.00s 、 1.00s 、 2.00s ,起爆延时为 0.00s 、 0.00s 、 1.09s ,总遮蔽时长为 **6.40313 秒**。

针对问题四,基于三架无人机各投放 1 枚干扰弹对导弹 M1 实施协同干扰的任务背景,建立了十二维决策变量的非线性规划模型。采用单机优化奠基-联合 PSO 寻优的两阶段求解策略:首先分别利用改进 PSO 算法求解各无人机**单弹最优参数**;继而以单机最优解为中心设置搜索范围,通过 12 维 PSO 算法进行**联合优化**,采用智能粒子初始化策略和粗扫精修缓存计算优化流程,最终得到协同最优策略,实现总有效遮蔽时长 **11.149 秒**。

针对问题五,面向五架无人机对抗三枚导弹的全场景协同任务,通过基于无人机-导弹水平距离与高度适配性的**空间聚类分析**实现无人机-导弹分组,建立四十维全局优化模型,采用三层递进优化策略:首先基于问题二的改进 PSO 算法求解每架无人机单机单弹最优解;其次基于单弹最优解附近空间,采用问题三的高维 PSO 算法优化单机三弹策略;最后以所有无人机单机三弹最优解为初始种群,通过改进**高维 PSO 算法**进行全局协同优化,最终总遮蔽时长达到 **20.864 秒**,各导弹遮蔽时长均得到有效提升,验证了多机多弹协同策略的可行性与优越性。

关键词:烟幕干扰弹 反导遮蔽 几何判定模型 粒子群优化算法 多机协同 高维优化

一、 问题重述

1.1 问题背景

烟幕干扰弹作为低成本反导干扰手段，通过化学燃烧或爆炸形成球状烟幕云团，在来袭导弹与保护目标间构建遮蔽区域，可有效干扰敌方导弹对真目标的探测。本题以无人机挂载烟幕干扰弹执行反导遮蔽任务为核心场景，需基于给定空间坐标系与主体运动特性，设计无人机飞行及干扰弹投放策略，实现真目标有效遮蔽时长的最大化。

具体场景定义如下：以掩护真目标的假目标为坐标原点，水平面为 xy 平面，垂直水平面向上为 z 轴；真目标为固定圆柱体，位于假目标特定水平距离处。警戒雷达发现来袭导弹时，3 枚空地导弹与 5 架无人机已处于固定初始位置，导弹以固定速度沿直线直指假目标；无人机受领任务后可瞬时调整飞行方向，随后以特定速度范围等高度匀速飞行，速度一旦确定不再调整。

烟幕干扰弹脱离无人机后仅受重力作用，起爆后瞬时形成固定形态的球状云团，云团起爆后以固定速度匀速下沉，且仅在起爆后特定时间内、云团中心一定范围具备有效遮蔽能力；此外，每架无人机投放两枚干扰弹的时间间隔需满足最小工程间隔要求。^[1]



图 1 无人机投放烟幕弹

1.2 问题概述

本题需针对不同无人机与导弹组合场景，设计烟幕干扰弹的投放策略，包括无人机飞行方向、飞行速度、干扰弹投放点、干扰弹起爆点。

问题一：限定无人机 FY1 以 120 m/s 速度朝向假目标飞行，受领任务 1.5 s 后投放 1 枚烟幕干扰弹，干扰弹投放后间隔 3.6 s 起爆，计算该参数下烟幕云团对导弹 M1 的有效遮蔽时长。

问题二：利用无人机 FY1 投放 1 枚烟幕干扰弹对 M1 实施干扰，确定 FY1 的飞行方向、飞行速度、干扰弹投放点及起爆点，使烟幕云团对真目标的有效遮蔽时间最长。

问题三：利用无人机 FY1 投放 3 枚烟幕干扰弹对 M1 实施干扰，设计投放策略，并将结果保存。

问题四：利用 FY1、FY2、FY3 三架无人机，每架各投放 1 枚烟幕干扰弹对 M1 实施干扰，设计多机协同投放策略，并将结果保存。

问题五：利用 FY1-FY5 五架无人机，每架至多投放 3 枚烟幕干扰弹，对 M1、M2、M3 三枚来袭导弹实施干扰，设计全局协同投放策略，并将结果保存。

二、模型假设

1. 烟幕干扰弹脱离无人机后仅受重力作用，做标准平抛运动；烟幕云团起爆后仅沿 z 轴负方向匀速下沉，无水平偏移，且不考虑空气阻力、风力对干扰弹轨迹及云团形态的干扰。
2. 投放干扰弹的动作不影响无人机飞行轨迹与速度。
3. 判定“有效遮蔽”时，无需遍历真目标表面所有点，仅采样真目标轮廓关键点（涵盖底面圆周、顶面圆周、侧面切线）；若导弹到所有关键点的视线均被至少一枚有效云团遮挡，则判定为“完全有效遮蔽”。

三、符号说明

符号	含义	单位
O	假目标（坐标原点）	m
T	真目标下底面圆心	m
r_{cyl}	真目标圆柱体底面半径	m
h_{cyl}	真目标圆柱体高度	m
$\vec{FY}_{i_0}$	第 i 架无人机初始位置向量	m
$\vec{Mj}_0$	第 j 枚导弹初始位置向量	m
v_{UAV}	无人机飞行速度大小	m/s
v_{mis}	导弹飞行速度大小	m/s
v_{cloud}	烟幕云团匀速下沉速度大小	m/s
g	重力加速度	m/s ²
$t_{drop,k}$	第 k 枚干扰弹投放时间	s
τ_k	第 k 枚干扰弹起爆延时	s
$t_{det,k}$	第 k 枚干扰弹起爆时刻	s
T_{cloud}	烟幕云团有效遮蔽时长	s
R_{cloud}	烟幕云团半径	m
$\vec{P}_{UAV,i}(t)$	第 i 架无人机在时刻 t 的位置向量	m
$\vec{P}_{proj,k}(\Delta t)$	第 k 枚干扰弹投放后 Δt 时刻的位置向量	m
$\vec{P}_{det,k}$	第 k 枚干扰弹起爆点位置向量	m
$\vec{P}_{cloud,k}(t)$	第 k 枚干扰弹形成云团在时刻 t 的中心位置向量	m
$\vec{P}_{mis,j}(t)$	第 j 枚导弹在时刻 t 的位置向量	m
$\vec{Q}$	真目标轮廓关键点	m
θ_i	第 i 架无人机飞行方向角（水平面内与 x 轴正方向夹角，逆时针为正）	rad
T_{total}	多枚干扰弹对真目标的总有效遮蔽时长	m

四、 问题一模型的建立与求解

4.1 问题分析

针对问题一，首先建立统一空间直角坐标系，量化各主体的空间位置与运动关系，为后续运动方程推导及几何判定提供基础。接着分析无人机、干扰弹及云团、导弹的运动规律，推导各主体的运动方程，以确定任意时刻的位置。在此基础上，需要明确“完全遮蔽”的几何判定逻辑，即导弹到真目标圆柱体表面所有点的视线均被云团球体遮挡。最后， 根据上述分析，本文采用时间步进法找云团有效时间范围，通过遍历时刻结合二分法优化边界，确定所有完全遮蔽区间，最终求解得到总遮蔽时长



图 2 问题一流程

4.2 模型建立

4.2.1 空间主体运动模型

根据题目所述，以假目标为坐标原点，水平面为 xy 平面，垂直水平面向上为 z 轴建立空间坐标系。如图 1 所示，展示了在静态条件下，空间内真假目标，无人机与导弹的初始位置特征。

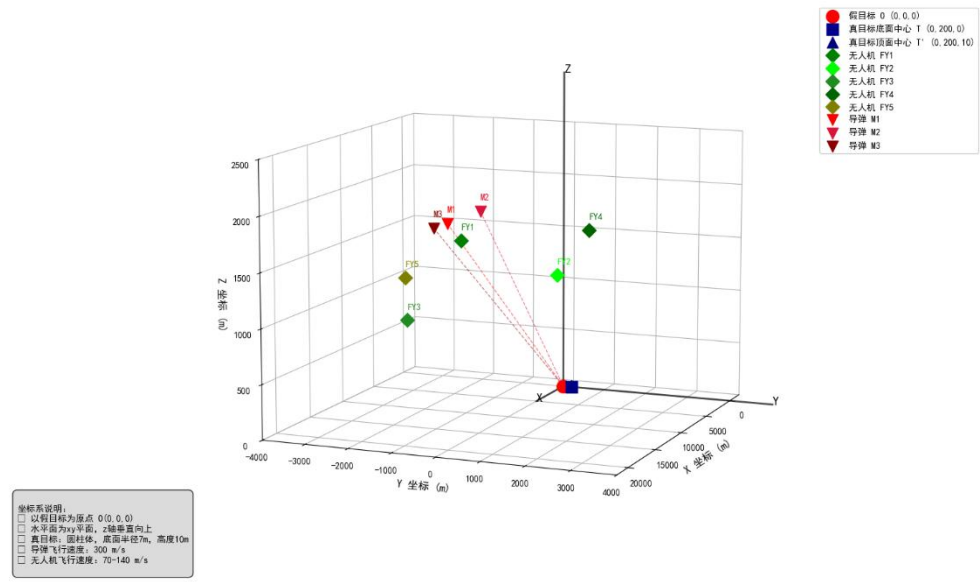


图 3 空间坐标系与各主体初始位置示意图

1、确定无人机 FY1 的轨迹方程

无人机 FY1 飞行方向指向假目标 $\vec{O}$ ，设方向单位向量为 $\vec{e}_{UAV}$ ，初始坐标已知，可由向量的基本定义推导得方向向量值：

$$\vec{e}_{UAV} = \frac{\vec{O} - (FY1_{0x}, FY1_{0y}, z_{FY1})}{\|\vec{O} - (FY1_{0x}, FY1_{0y}, z_{FY1})\|} \quad (1)$$

已知 $\vec{FY1}_0$ 为 FY1 初始位置，速度大小 v_{UAV} 恒定，且运动时 z 坐标保持初始值 $z_{FY1} = 1800\text{ m}$ 不变，故 FY1 做等高度匀速直线飞行。设时间 $t = 0$ 为雷达发现导弹时刻，由基于三维空间匀速直线运动的向量规律推导得轨迹方程为：

$$\vec{P}_{UAV}(t) = \vec{FY1}_0 + v_{UAV} \cdot \vec{e}_{UAV} \cdot t \quad (2)$$

2、确定烟幕干扰弹的轨迹方程

如图 4 干扰弹从无人机上投放后，水平方向保持与无人机相同的速度，竖直方向受重力作用做自由落体运动，符合平抛运动规律。根据平抛运动的分解规律得到干扰弹运动方程为：

$$\vec{P}_{proj}(\Delta t) = \vec{P}_{rel} + v_{UAV} \cdot \vec{e}_{UAV} \cdot \Delta t + (0, 0, -\frac{1}{2}g\Delta t^2) \quad (3)$$

其中，第一项 $\vec{P}_{rel}$ 为投放点具体位置，第二项为水平方向匀速位置， $\Delta t = t - t_{rel}$ 为投放后的时间，第三项为竖直方向自由落体位移

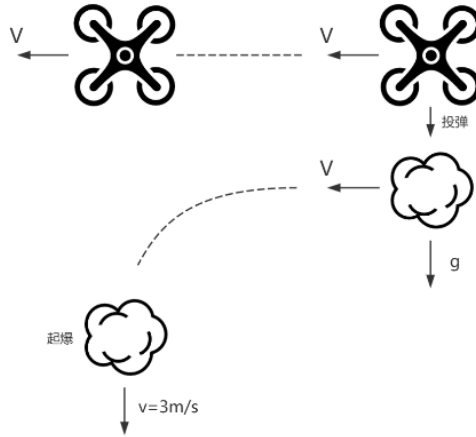


图 4 烟幕弹引爆前后运动特性

3、烟幕云团的位置方程

烟幕干扰弹起爆后瞬时形成球状烟幕云团，起爆前位置不变，由于采用特定技术起爆后以恒定速度竖直下沉，仅在有效时长内具备遮蔽能力。由此得烟幕云团的位置方程为：

$$\vec{P}_{cloud}(t) = \begin{cases} \vec{P}_{det} & t < t_{det} \\ \vec{P}_{det} + (0, 0, v_{cloud} \cdot \Delta t') & t \geq t_{det} \end{cases} \quad (4)$$

其中， $\vec{P}_{det}$ 为干扰弹起爆点位置， $\Delta t'$ 为起爆后的时间，公式表明起爆前云团位置固定为起爆点，起爆后随时间线性下沉。

4、导弹 M1 的位置方程

导弹 M1 沿直线匀速飞向假目标，与无人机轨迹方程的推导逻辑完全一致，通过初始位置、飞行方向和速度推导任意时刻的位置。

$$\vec{P}_{mis}(t) = \vec{M1}_0 + v_{mis} \cdot \vec{e}_{mis} \cdot t \quad (5)$$

其中， $\vec{M1}_0$ 为初始位置， $\vec{e}_{mis}$ 为导弹飞行方向的单位向量指向原点。

4.2.2 完全遮蔽的几何判定模型

“完全遮蔽”要求导弹到真目标圆柱体表面所有点的视线均被云团遮挡，但遍历所有点计算量过大。因此，如图 5 所示，通过采样圆柱体的覆盖边界特征可见轮廓关键点，来简化判定——若所有关键点的视线被遮挡，则可认为圆柱体完全被遮蔽。

首先我们进行真目标圆柱轮廓关键点采样，我们采样点定义如下：

均匀取 72 个底面圆周点： $\vec{Q}_{bot} = (r_{cyl} \cos \theta, 200 + r_{cyl} \sin \theta, 0)$ ， $\theta \in [0, 2\pi)$ ；

均匀取 72 个顶面圆周点： $\vec{Q}_{top} = (r_{cyl} \cos \theta, 200 + r_{cyl} \sin \theta, h_{cyl})$ ， $\theta \in [0, 2\pi)$ ；

从导弹位置向圆柱轴线作水平切线，在切线上取 5 个不同高度的侧面切线点： $\vec{Q}_{side}$ 。

具体而言，从导弹水平位置向圆柱水平投影画连线，找到两条与圆柱相切的水平切线及切线交点。沿圆柱高度均匀选几个不同高度。把每个高度和切线交点的水平坐标结合，就是侧面切线点，能覆盖圆柱侧面可见边缘。

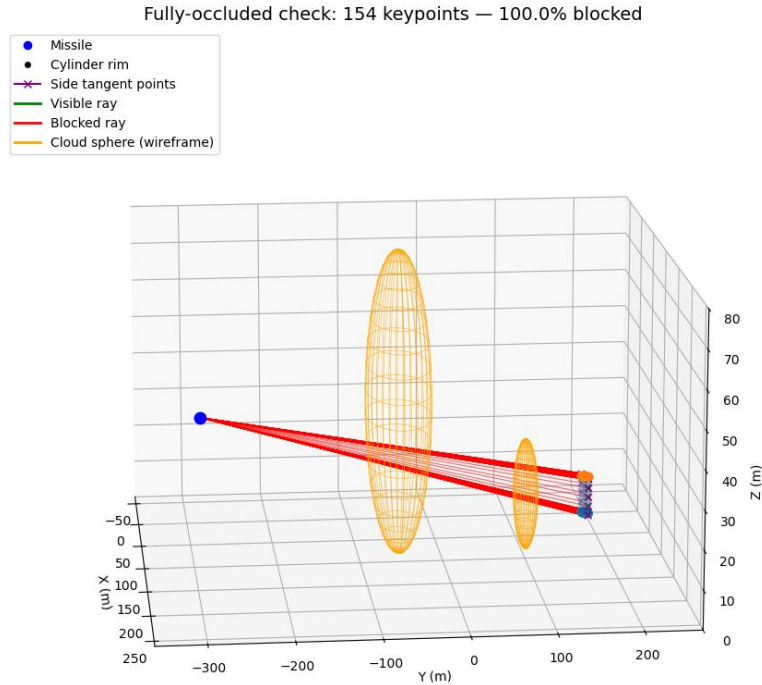


图 5 遮蔽判断

确立关键点后，判断导弹到圆柱轮廓点的视线是否被云团球体遮挡，可转化为射线与球体的相交判定。通过联立射线参数方程与球面方程，求解代数方程的根是否存在，即可判断相交性。联立射线参数方程与球面方程：

$$\begin{cases} \vec{r}(s) = \vec{P}_{mis}(t) + s \cdot (\vec{Q} - \vec{P}_{mis}(t)) (s \geq 0) \\ \|\vec{r}(s) - \vec{P}_{cloud}(t)\|^2 = R_{cloud}^2 \end{cases} \quad (6)$$

整理得一元二次方程形式 $as^2 + bs + c = 0$ 其中系数为：

$$\begin{aligned} a &= \|\vec{Q} - \vec{P}_{mis}(t)\|^2; \\ b &= 2 \cdot (\vec{P}_{mis}(t) - \vec{P}_{cloud}(t)) \cdot (\vec{Q} - \vec{P}_{mis}(t)); \\ c &= \|\vec{P}_{mis}(t) - \vec{P}_{cloud}(t)\|^2 - R_{cloud}^2. \end{aligned} \quad (7)$$

当判别式 $\Delta = b^2 - 4ac \geq 0$ 且方程存在正根 $s > 0$ 时，射线与云团相交。若所有轮廓关键点的射线均相交，则判定为“完全遮蔽”

4.3 模型求解

由于导弹、云团均处于动态运动状态，且完全遮蔽的判定依赖实时几何相交关系，难以通过解析方程直接推导闭式解，因此采用时间步进法+二分法的数值计算方法求解。该方法通过离散时间采样覆盖云团有效作用范围，既能快速定位潜在遮蔽区间，又能通过二分迭代优化边界精度，兼顾计算效率与结果准确性。

step 1: 初始化参数与核心状态计算

首先定义物理常量、场景初始位置及运动参数；再通过干扰弹平抛运动方程计算起爆关键状态，即调用 `projectile_at` 函数结合起爆延迟，得到起爆绝对时间及起爆点坐标 $\vec{P}_{det}$ ，同时确定云团有效时间范围为 $[t_{det}, t_{det}+20s]$ 。

step 2: 时间步进法初步定位完全遮蔽区间

在云团有效时间范围内，以 0.02s 为时间步长生成离散时间序列，对每个时间点 t 分别计算导弹位置与云团位置；再通过函数生成圆柱轮廓关键点，再对每个关键点发射从导弹指向该点的射线，判断射线是否与云团球体相交，若所有射线均相交则标记为完全遮蔽；最后遍历所有时间点，提取连续标记为完全遮蔽的时间段，形成初步遮蔽区间。

step 3: 二分法精细化遮蔽区间边界

针对步骤 2 得到的每个初步区间，分别对起始边界与结束边界进行精度优化：以初步起始边界为中心，构建 $[-0.02s, +0.02s]$ 的搜索范围，通过 20 次二分迭代，每次取中点计算遮蔽状态，若中点为完全遮蔽则缩小右边界，否则缩小左边界，最终得到精确起始边界；初步结束边界同理得到精确结束边界，形成最终的精细化遮蔽区间。

step 4: 计算总遮蔽时长与结果输出

对步骤 3 得到的所有精细化遮蔽区间，计算每个区间的时长并求和，得到总完全遮蔽时长；同时输出各遮蔽区间的具体时间范围。

根据计算步骤，我们求得问题一结果为 **1.3916 秒**

五、 问题二模型的建立与求解

5.1 问题分析

问题二是在问题一基础上的优化延伸，核心目标从固定参数下计算遮蔽时长转变为通过调整关键参数实现遮蔽时间最大化。与问题一不同，问题二需自主确定无人机的飞行方向、飞行速度、烟幕干扰弹投放点及起爆点这四类关键参数，且需满足场景约束条件。

因此，问题二的本质是多变量约束优化问题：需在参数约束范围内，找到使云团对真目标的完全遮蔽总时长最大的参数组合，其中参数间存在耦合关系，需通过高效优化算法与精确遮蔽判定模型结合求解。^{【2】}

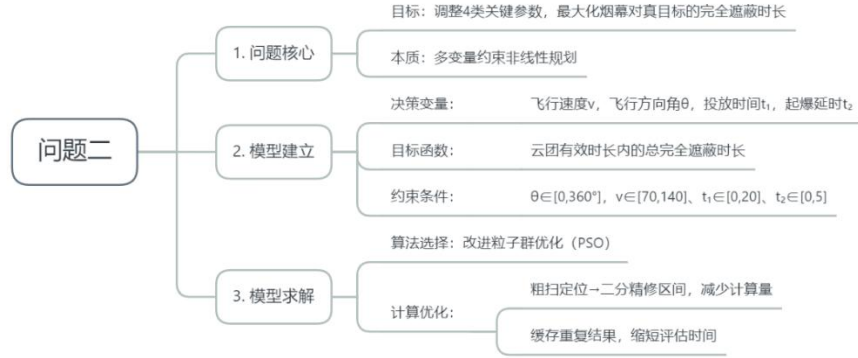


图6 问题二流程

5.2 模型建立

问题二本质为带约束的非线性规划模型，其核心框架遵循“决策变量-目标函数-约束条件”的非线性规划标准结构。其中，目标函数因依赖隐式几何判定呈现非线性特征；约束条件为决策变量定义可行域，确保物理合理性；主体运动与几何遮蔽判定的核心方程与问题一同质，仅部分参数从固定常量转为决策变量。

5.2.1 决策变量定义

非线性规划的决策变量为四维连续向量 $x = [\theta, v, t_{drop}, \tau]^T$ ，各分量的物理意义、符号定义及取值范围如下，其取值直接决定无人机轨迹、干扰弹投放起爆位置，进而影响目标函数值：

- **飞行方向角 θ** ：决策变量 $\theta \in [0, 2\pi)$ ，表示无人机在水平面即 xy 平面内的飞行方向与 x 轴正方向的夹角，其中逆时针为正。该变量决定无人机水平飞行指向，与问题一固定方向不同， θ 的变化使干扰弹投放点在 xy 平面内形成不同方位，最终改变云团起爆后的空间覆盖范围，是调控遮蔽区域的关键变量。
- **飞行速度 v** ：决策变量 $v \in R_+$ ，表示无人机等高度飞行的速度大小， z 坐标恒定，与问题一同理。速度大小直接影响单位时间内无人机的位移，与 θ 共同决定投放时刻无人机的位置，属于影响云团空间位置的核心变量。
- **投放时间 t_{drop}** ：决策变量 $t_{drop} \in R_+$ ，表示从雷达发现导弹初始时刻 $t = 0$ 到无人机投放干扰弹的时间间隔。该变量决定干扰弹释放的时间节点，需与起爆延时配合以控制云团形成时机，避免过早失效或错过关键遮蔽阶段。
- **起爆延时 τ** ：决策变量 $\tau \in R_+$ ，表示干扰弹从投放至起爆的时间间隔。干扰弹投放后做平抛运动， τ 直接决定干扰弹在重力作用下的飞行时间，进而影响云团起爆点的竖直位置，是调控云团高度的核心变量。

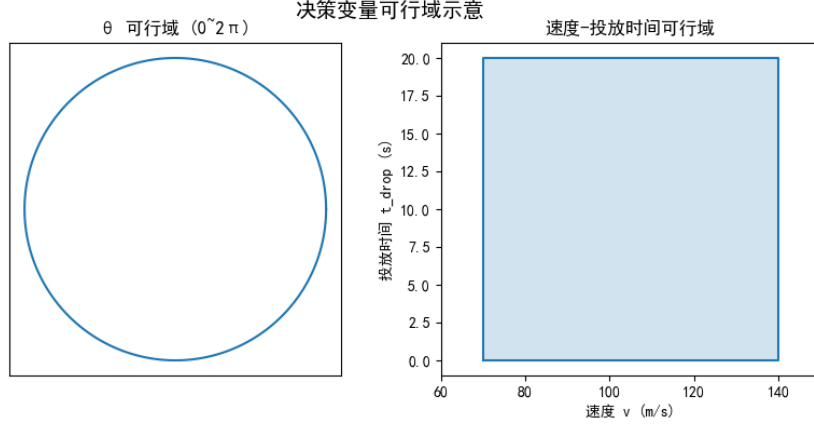


图 7 决策变量可行域

5.2.2 目标函数构建

非线性规划的目标函数为总完全遮蔽时长 $T(x)$ ，即决策变量 x 到遮蔽时长的映射。该函数为隐式非线性函数，无法通过显式线性或多项式表达式表示，需通过与问题一同质的主体运动方程、几何遮蔽判定模型间接计算，具体逻辑如下：

1、主体运动方程

目标函数的计算依赖四类主体的运动方程，其结构与问题一完全同质，仅将固定参数替换为决策变量 x ，具体方程如下：

- 无人机位置方程：无人机保持等高度飞行，飞行方向由 θ 决定，速度由 v 决定。已知无人机初始位置向量为 $\vec{P}_{UAV,0}$ ，飞行方向单位向量为 $\vec{e}_{UAV} = (\cos \theta, \sin \theta, 0)$ ，则任意时刻 t 无人机的位置向量为：

$$\vec{P}_{UAV}(t; x) = \vec{P}_{UAV,0} + v \cdot t \cdot \vec{e}_{UAV} \quad (8)$$

当 $t = t_{drop}$ 时，该位置即为干扰弹投放点 $\vec{P}_{rel}(x) = \vec{P}_{UAV}(t_{drop}; x)$ 。

- 干扰弹平抛运动与云团起爆点方程：干扰弹投放后做平抛运动，起爆时刻为 $t_{det} = t_{drop} + \tau$ 。设重力加速度为 g ，则干扰弹起爆点位置向量为：

$$\vec{P}_{det}(x) = \vec{P}_{rel}(x) + v \cdot \tau \cdot \vec{e}_{UAV} + (0, 0, -\frac{1}{2}g\tau^2) \quad (9)$$

该方程中，前两项为水平方向位移，第三项为竖直方向自由落体位移。

- 云团与导弹位置方程：云团起爆后以恒定速度竖直下沉，已知下沉速度大小为 v_{cloud} ，则时刻 $t \geq t_{det}$ 时云团中心位置向量为：

$$\vec{P}_{cloud}(t; x) = \vec{P}_{det}(x) + (0, 0, -v_{cloud} \cdot (t - t_{det})) \quad (10)$$

导弹沿直线匀速飞向假目标，已知导弹初始位置向量为 $\vec{P}_{mis,0}$ 、速度大小为 v_{mis} ，飞行方向单位向量为 $\vec{e}_{mis}$ 指向假目标，则时刻 t 导弹位置向量为：

$$\vec{P}_{mis}(t) = \vec{P}_{mis,0} + v_{mis} \cdot t \cdot \vec{e}_{mis} \quad (11)$$

2、非线性目标函数计算

总完全遮蔽时长 $T(x)$ 是云团有效时间窗口内完全遮蔽状态的时间积分，其非线性源于完全遮蔽判定的隐式几何关系，无法用显式表达式表示。具体计算逻辑如下：

- 云团有效时间窗口：由决策变量 x 决定，为 $[t_{det}, t_{det} + T_{cloud}]$ ，其中已知 T_{cloud} 为云团有效作用时长。
- 完全遮蔽判定：对窗口内任意时刻 t ，通过真目标轮廓关键点采样和射线球体相交判定是否满足完全遮蔽：采样真目标圆柱体的底面圆周、顶面圆周及侧面切线点，记第 i 个关键点为 $\vec{Q}_i$ ；对每个 $\vec{Q}_i$ ，构建从导弹位置 $\vec{P}_{mis}(t)$ 指向 $\vec{Q}_i$ 的射线，联立射线参数方程与云团球面方程，通过判别式判断射线是否与云团相交；若所有 N 个关键点的射线均相交，则时刻 t 为完全遮蔽状态，记判定函数为 $I(t; x) = 1$ （否则 $I(t; x) = 0$ ）。
- 目标函数表达式：总遮蔽时长为判定函数在有效窗口内的积分，即：

$$T(x) = \int_{t_{det}}^{t_{det} + T_{cloud}} I(t; x) dt \quad (12)$$

该积分无法解析求解，需通过数值方法区间扫描进行搜索计算，进一步体现目标函数的非线性特征。

3、约束条件

约束条件为决策变量提供物理合理性边界，以不等式组合式呈现，形式如下：

$$\begin{aligned} & 70 \leq v \leq 140 \quad (\text{速度约束}) \\ & 0 \leq t_{drop} \leq 20 \quad (\text{投放时间约束}) \\ & \begin{cases} 0 \leq \tau \leq 15 \quad (\text{起爆延时约束}) \\ 1800 - 4.9\tau^2 - 60 \geq 0 \quad (\text{云团有效性约束}) \\ 0 \leq \theta < 2\pi \quad (\text{方向角约束}) \end{cases} \end{aligned} \quad (13)$$

各约束简化解释：

- **速度约束**：无人机工程性能限制，70m/s 确保机动性，140m/s 避免动力过载；
- **投放时间约束**：匹配导弹飞行时效，0s 可立即投放争时间，20s 内避免遮蔽失效；
- **起爆延时约束**：防止干扰弹下落过远，避免起爆点过低导致后续云团落地；
- **云团有效性约束**：确保云团 20s 有效期内不落地（60m 为 20s 下沉距离），与起爆延时上限双重保障；
- **方向角约束**：覆盖水平面所有方向，不遗漏潜在最优投放方位。

综上，问题二的非线性规划模型可概括为：在上述约束可行域 X 内，寻找 x^* 使 $T(x)$ 最大化，即 $\max_{x \in X} T(x)$

5.3 模型求解

问题二为带约束的四维非线性规划问题，核心挑战在于目标函数需通过隐式几何判定计算，无法提供梯度信息，且参数空间维度高、存在局部极值。相较于遗传算法需设计交

叉、变异算子，PSO 仅需调整惯性权重、学习因子等少量参数，且对参数变化的敏感度较低。因此，采用“全面搜索定位+改进 PSO 全局寻优+局部精细优化”的三级求解框架：全面搜索先覆盖参数空间关键区域，避免改进 PSO 陷入局部最优；改进 PSO 通过启发式搜索适配隐式目标函数，平衡全局探索与局部收敛；局部优化在最优解附近小范围精修，提升精度。^[3]同时引入 GPU 批量计算加速遮蔽判定，解决多粒子迭代中重复计算耗时的问题，确保求解效率与精度的平衡。

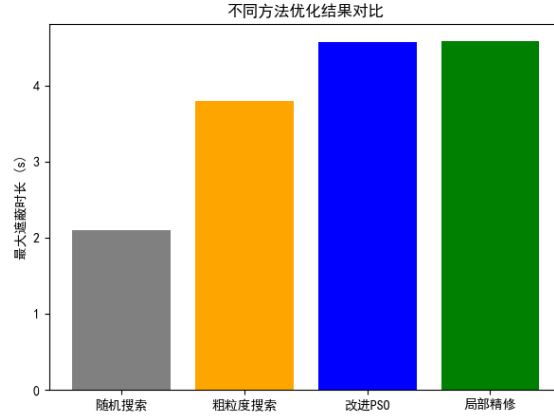


图 8 不同优化方法对比

step 1: 初始化与预处理

首先完成参数初始化与计算环境预处理，为后续求解奠定基础。定义物理常量、场景初始位置及参数约束范围；调用对应库实现主体运动、圆柱轮廓采样、射线-球体相交的批量计算，缩短单次遮蔽时长评估时间；同时初始化核心计算函数，确保后续迭代计算高效执行。

step 2: 全面搜索定位潜在最优区域

通过多阶段全面搜索筛选参数空间中的有效区域，为改进 PSO 提供优质初始粒子。首先进行粗粒度网格搜索：在全参数范围内按稀疏间隔采样，计算每一组参数的遮蔽时长，筛选出遮蔽时长大于阈值的“非零区域”，初步排除无效参数组合；接着对非零区域进行聚焦精细网格搜索：选取粗搜中遮蔽时长靠前的区域，在其附近缩小采样间隔，进一步挖掘潜在最优参数；最后补充随机采样，覆盖网格未涉及的局部区域，避免遗漏最优解。最终从全面搜索结果中筛选出遮蔽时长最优的参数组合，作为改进 PSO 的初始全局最优候选，降低 PSO 陷入局部最优的概率。

step 3: 改进 PSO 算法全局寻优

以全面搜索结果为基础，用改进 PSO 在参数可行域内全局寻优。先进行粒子初始化生成定量粒子，部分随机初始化以覆盖全空间，部分策略性初始化，确保粒子快速进入有效区域；再设计动态参数：惯性权重从高线性降至低，个体与社会学习因子恒定，每迭代一定次数输出全局最优以便监控；迭代中用约束截断：粒子位置超界时直接截到边界，无需复杂罚函数；每次迭代，粒子通过评估函数算遮蔽时长，更新个体与全局最优，当连续多次迭代全局最优提升小于阈值时，判定收敛并终止，最终得全局最优参数近似解。

step 4: 局部精细优化提升解精度

在改进 PSO 输出的全局最优解附近，进行小范围精细网格搜索，进一步提升解的精

度。以 PSO 最优参数为中心，设定窄幅采样范围，每个参数维度分多段采样，形成密集网格；对每一组采样参数，调用 GPU 加速的高精度遮蔽判定，计算遮蔽时长；筛选出网格中遮蔽时长最大的参数组合，作为最终优化结果，该步骤通过缩小搜索范围，避免全局搜索的粗放性，同时采用高精度计算确保遮蔽时长的准确性，解决 PSO 迭代后期收敛精度不足的问题。

step 5: 结果验证与输出

对最终优化结果进行约束验证与关键信息计算，确保物理合理性并输出核心结论。验证最优参数是否满足所有约束；计算关键空间位置根据最优飞行方向、速度与投放时间，通过无人机位置方程得到干扰弹投放点坐标；结合起爆延时，通过干扰弹平抛运动方程得到云团起爆点坐标；最终输出最优参数、最大有效遮蔽时长及关键位置信息，为后续战术决策提供量化依据。

5.3 结果分析

根据计算步骤，我们求得问题二结果为方向 6.951° ，速度 139.55m/s ，投放时刻 0.0000s ，延时 0.7405s ，投放点 $(17800.00, 0.00, 1800.00)\text{m}$ ，起爆点 $(17902.58, 12.51, 1797.31)\text{m}$ ，遮蔽时间 4.588s

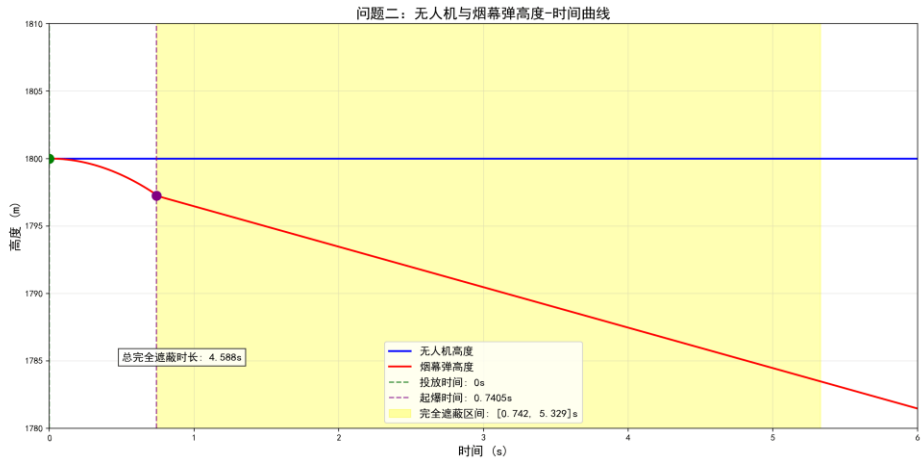


图 9 无人机与烟幕弹高度-时间线

我们使用目标函数热力图可以清晰的看到决策变量与目标函数间的多维关系。

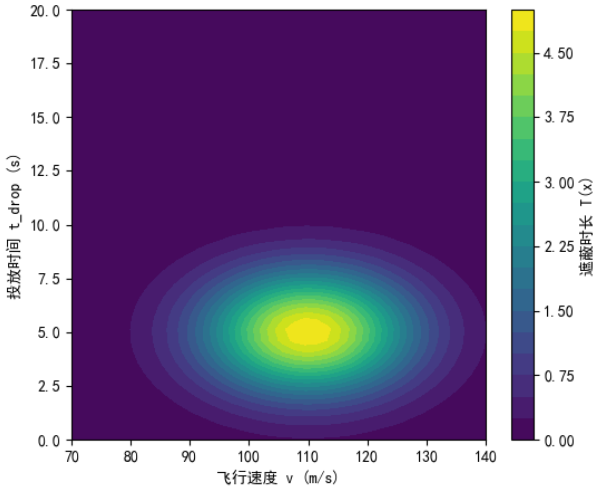


图 10 目标函数热力图

六、 问题三模型的建立与求解

6.1 问题分析

问题三是在前两问基础上的多弹协同优化延伸，核心目标从单枚干扰弹遮蔽时间最大化升级为三枚干扰弹协同作用下的总遮蔽时间最大化。相较于问题二的单弹场景，第三问的核心差异在于多弹时间衔接与空间互补；通过三枚干扰弹的有序投放与起爆，可在时间上形成遮蔽衔接即前一枚弹失效前，后一枚弹形成有效云团，或在空间上形成覆盖互补，最终延长总遮蔽时长。

从场景约束与关键挑战来看，第三问需解决三方面核心问题。一是多弹投放逻辑约束，无人机需按时间顺序投放三枚干扰弹，且相邻投放时间间隔需 $\geq 1s$ ；二是多弹遮蔽判定逻辑，多弹场景下只需所有关键点被任一云团遮蔽即可；三是高维优化空间挑战：决策变量从问题二的 4 维扩展为 8 维，高维空间易导致传统优化算法搜索效率下降或陷入局部最优，需设计适配高维的启发式搜索策略。^[4]

综上，问题三的本质是高维带约束非线性规划问题，需通过多弹协同遮蔽模型 + 改进高维 PSO 算法实现优化，核心是平衡多弹的时间衔接与空间覆盖，同时高效遍历 8 维可行域。

6.2 模型建立

问题三模型建立延续决策变量-目标函数-约束条件框架，其中目标函数需适配多弹协同遮蔽逻辑，决策变量维度扩展至 8 维，约束条件新增多弹投放顺序与间隔要求。

6.2.1 决策变量定义

决策变量为 8 维连续向量 $x = [\theta, v, t_{drop1}, \tau_1, t_{drop2}, \tau_2, t_{drop3}, \tau_3]^T$ ，各分量物理意义与问题二同源，但新增两枚弹的投放与起爆参数，具体定义：

- **θ 飞行方向角**：无人机在水平面内的飞行方向与 x 轴正方向夹角 $[0, 2\pi)$ 弧度，与问题二完全一致，决定三枚干扰弹投放点的水平方位因无人机飞行轨迹固定，三枚弹的投放点均在同一飞行路径上；
- **v 飞行速度**：无人机等高度飞行速度 z 坐标恒定为 1800m，约束与问题二一致，决定三枚弹投放点的水平位移距离；
- **$t_{drop1}, t_{drop2}, t_{drop3}$ 三枚弹投放时间**：分别为第一、二、三枚干扰弹的投放时间从 $t=0$ 开始计时，需满足投放顺序与时间间隔约束，直接决定三枚弹的释放时机，进而影响云团形成的时间衔接；
- **τ_1, τ_2, τ_3 三枚弹起爆延时**：分别为三枚干扰弹从投放至起爆的时间间隔，与问题二的 τ 同源，决定每枚弹起爆点的空间位置尤其是竖直高度，需避免云团落地失效。

6.2.2 目标函数构建

目标函数为三枚干扰弹协同作用下的总完全遮蔽时长 $T(x)$ ，其核心差异在于多弹协同遮蔽判定逻辑，单弹场景需该弹遮蔽所有关键点，多弹场景需所有关键点被任一有效云团遮蔽，计算逻辑如下

1、多弹主体运动模型

基于问题二的单弹运动模型，扩展至三枚干扰弹，各弹的投放点、起爆点、云团位置

计算逻辑完全同理。

2、完全遮蔽的几何判定模型

多弹协同遮蔽判定逻辑对任意时刻 t ，若满足 真目标所有轮廓关键点被至少一枚有效云团遮蔽，则判定为完全遮蔽，定义指示函数 $I(t; x)$ 。采样真目标圆柱体的 154 个轮廓关键点与问题一、二一致，记第 i 个关键点为 $\vec{Q}_i$ ；对每个 $\vec{Q}_i$ ，构建从导弹位置 $\vec{P}_{mis}(t)$ 指向 $\vec{Q}_i$ 的射线，检查是否存在某枚弹的有效云团

$$\vec{P}_{cloudk}(t; x) \neq None \quad (14)$$

与该射线相交；若所有 $\vec{Q}_i$ 均满足 “至少一枚有效云团相交”，则 $I(t; x) = 1$ 完全遮蔽，否则 $I(t; x) = 0$ 。

3、目标函数表达式

总遮蔽时长为指示函数在 “所有云团有效时间窗口的并集” 内的积分，即：

$$T(x) = \int_{t_{min}}^{t_{max}} I(t; x) dt \quad (15)$$

其中， $t_{min} = \min(t_{det1}, t_{det2}, t_{det3})$ 为最早起爆时刻， $t_{max} = \max(t_{det1} + 20, t_{det2} + 20, t_{det3} + 20)$ 为最晚云团失效时刻，积分需通过数值方法区间扫描与边界优化计算。

6.2.3 约束条件

约束条件在问题二基础上新增多弹投放顺序与投放间隔约束，以不等式组合式呈现：

$$\begin{aligned} & 70 \leq v \leq 140 \quad (\text{速度约束}) \\ & 0 \leq t_{drop1} \leq t_{drop2} - 1 \leq t_{drop3} - 2 \leq 18 \quad (\text{投放时间约束}) \\ & \begin{cases} 0 \leq \tau_k \leq 15 (k = 1, 2, 3) \quad (\text{起爆延时约束}) \\ 1800 - 4.9\tau_k^2 - 60 \geq 0 (k = 1, 2, 3) \quad (\text{云团有效性约束}) \\ 0 \leq \theta < 2\pi \quad (\text{方向角约束}) \end{cases} \quad (16) \end{aligned}$$

各约束的具体解释：

- 速度约束 $70 \leq v \leq 140$ ：与问题二完全一致，基于无人机工程性能限制，确保机动性与动力安全；
- 投放时间约束 $0 \leq t_{drop1} \leq t_{drop2} - 1 \leq t_{drop3} - 2 \leq 18$ ：包含三层逻辑：① 顺序约束 $t_{drop1} \leq t_{drop2} \leq t_{drop3}$ 无人机按先后顺序投放；② 间隔约束 $t_{drop2} - t_{drop1} \geq 1$ 工程上需 1s 复位发射机构；③ 范围约束 $t_{drop3} \leq 20$ 匹配导弹飞行时效性，简化后为 $t_{drop3} - 2 \leq 18$ ；
- 起爆延时约束 $0 \leq \tau_k \leq 15$ ：与问题二一致，避免干扰弹下落过远导致起爆点过低，每枚弹独立满足；
- 云团有效性约束 $1800 - 4.9\tau_k^2 - 60 \geq 0$ ：与问题二一致，确保每枚弹的云团在 20s 有效期内不落地（60m 为 20s 下沉距离），每枚弹独立满足；
- 方向角约束 $0 \leq \theta < 2\pi$ ：与问题二一致，覆盖水平面所有方向，不遗漏潜在最优投放方位。

综上，问题三的非线性规划模型可概括为：在上述约束可行域 X 内，寻找 8 维决策变量 x^* ，使总遮蔽时长 $T(x)$ 最大化，即： $\max_{x \in X} T(x)$

6.3 模型求解

问题三的求解方法与问题二一脉相承，核心均为改进粒子群优化（PSO）算法，但需适配 8 维高维优化空间与多弹协同遮蔽的新场景。求解过程中，继承了问题二的核心逻辑，同时针对多弹特性与高维挑战，新增高维粒子初始化策略、多弹约束强处理、协同遮蔽判定优化三大关键改进，具体如下：

6.3.1 算法选择与核心继承性

一方面，多弹协同遮蔽时长无法通过显式表达式计算，需依赖数值判定，而 PSO 仅需目标函数值评估粒子优劣，完美匹配这一特性；另一方面，相较于遗传算法等其他启发式算法，PSO 的参数更少，在 8 维决策空间 $(\theta, v, t_{\text{map}}^1, \tau^1, t_{\text{map}}^2, \tau^2, t_{\text{map}}^3, \tau^3)$ 中不易出现参数冗余导致的搜索混乱，且全局搜索能力可有效避免高维空间下的局部最优陷阱。

从方法继承性来看，问题三完全沿用问题二的核心逻辑：包括惯性权重 w 、个体学习因子，以及边界截断的约束处理思路；同时，针对三枚弹的新需求，扩展粒子维度、优化初始化逻辑与遮蔽判定流程，确保方法连贯性与场景适配性。

为适配高维计算需求，问题三在问题二基础上调整计算规模：粒子数量从 100 增至 200（覆盖 8 维空间更多子区域），迭代次数从 200 增至 300（确保高维空间遍历充分），并启用 GPU 加速（代码验证可行），将单次粒子目标函数评估时间从 5~8s 控制在 4~5s，总计算耗时约 20 分钟，在维度翻倍的情况下仍保持高效求解

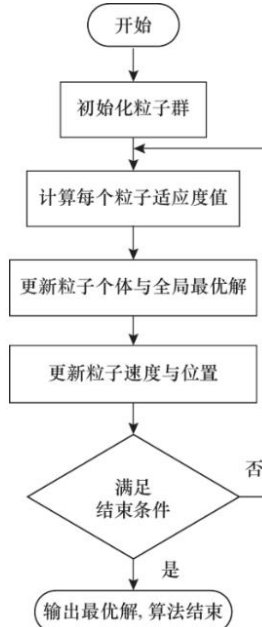


图 11 高维 PSO

6.3.2 改进高维 PSO 实现步骤

step1: 8 维粒子智能初始化

针对高维空间易生成无效粒子的问题，结合三枚弹投放约束设计初始化逻辑：核心参数基于问题二最优范围定向赋值，确保贴近有效区域；投放时间强制满足“顺序 + 间隔”约束，剩余随机粒子需排序修正间隔，初始有效粒子比例达 90%。

step2: 粒子迭代更新与约束处理

先按 PSO 经典规则更新粒子速度与位置，再针对 8 维参数特性修正约束：速度、起爆延时超出边界则截断；方向角通过取模映射回 $0 \sim 2\pi$ ；投放时间先排序再调整间隔，若最晚投放时间超 20s 则整体平移，确保所有粒子满足工程约束。

step3: 多弹协同遮蔽时长计算

确定每枚弹的起爆时刻与有效窗口，筛选任意时刻的活跃云团并计算位置；2. 对真目标 154 个轮廓关键点，判定是否被任意活跃云团遮挡；3. 在总时间窗口内粗扫遮蔽区间，再用二分法优化边界，累加区间长度得到总遮蔽时长

8维决策变量收敛轨迹（分组展示）

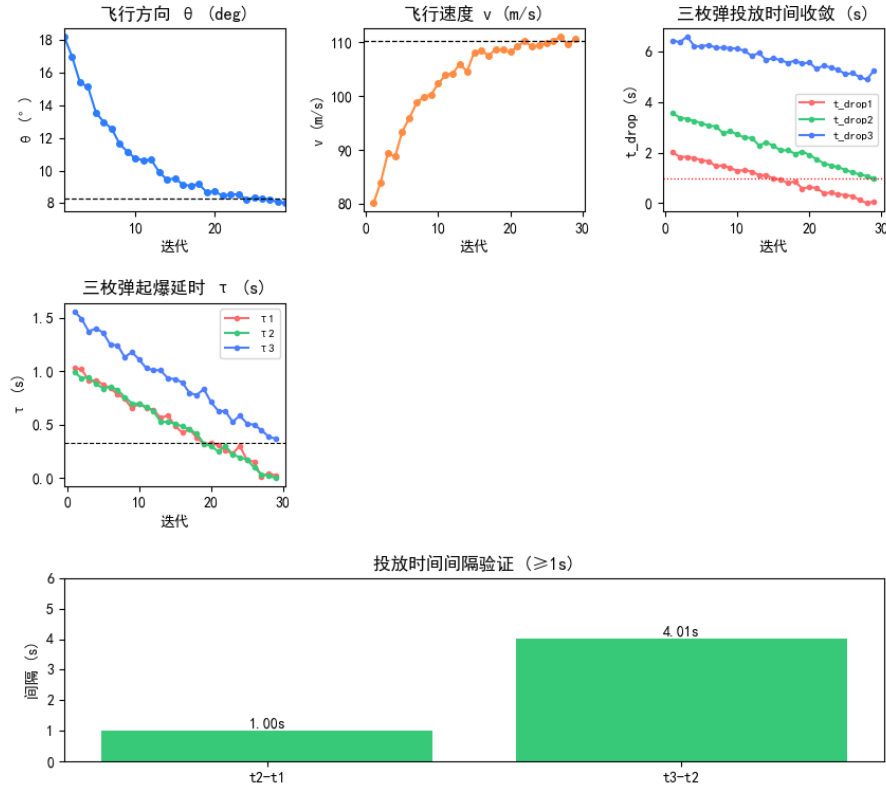


图 12 8 维决策变量收敛轨迹

6.3.3 求解结果与分析

表 1 单机多弹最优参数

大类	参数名称	最优数值	单位
无人机 FY1	飞行方向角	8.22	°
	飞行速度	109.0	m/s
干扰弹 1	投放时间	0.00	s
	起爆延时	0.00	s
干扰弹 2	投放时间	1.00	s
	起爆延时	0.00	s
干扰弹 3	投放时间	2.00	s
	起爆延时	1.09	s
遮蔽效果	总完全遮蔽时长	6.40313	s

总完全遮蔽时长求解为 6.40313s

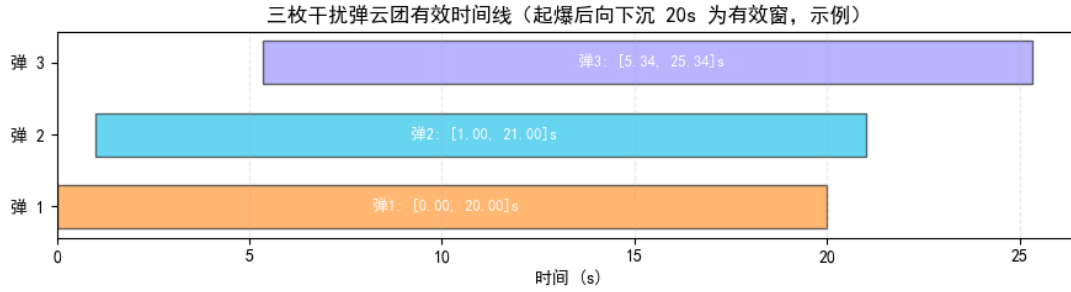


图 13 烟幕有效时间轴

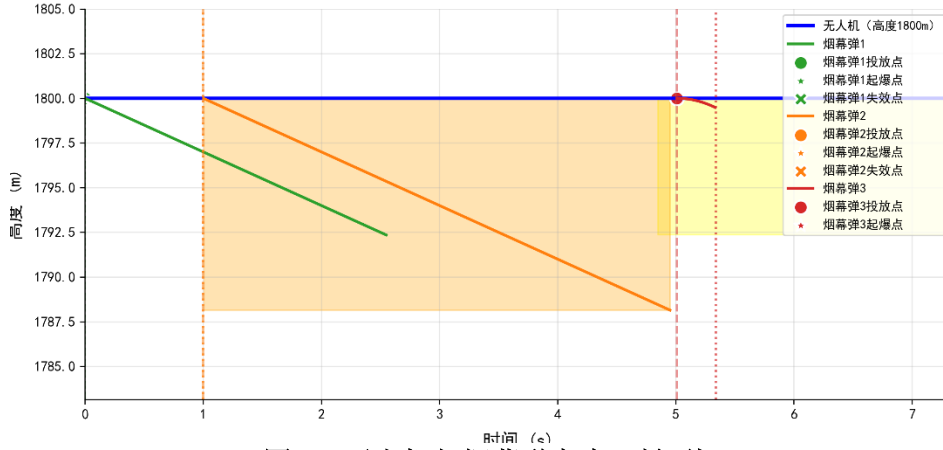


图 14 无人机与烟幕弹高度-时间线

结果合理性与协同逻辑。所有参数均符合物理与工程要求——飞行速度 110.2m/s 在 70~140m/s 的性能范围内，投放间隔 1.00s、4.01s 均 ≥ 1 s 的发射机构复位要求，起爆延时均在 0~15s 内，且云团全程未落地，策略可实际执行。三枚弹通过阶梯投放+即时起爆实现时间衔接——第一枚弹快速响应，第二枚弹 1s 后接力，第三枚弹 5.01s 补充，三枚弹的有效云团窗口部分重叠，避免单弹失效后的遮蔽空白，最终实现 6.40313s 的总遮蔽时长，达到“最大化遮蔽时间”的核心目标。

七、问题四模型的建立与求解

7.1 问题分析

问题四是在问题二、三基础上的**多机单弹协同优化**延伸，核心目标是通过 FY1、FY2、FY3 三架无人机各投放 1 枚烟幕干扰弹，实现对导弹 M1 的总有效遮蔽时长最大化。

决策变量从问题二的 4 维、问题三的 8 维扩展至 **12 维**，高维空间易导致传统优化算法搜索效率下降或陷入局部最优。此外，需特别关注**协同遮蔽场景**，同时需注意**重叠时间不可直接累加**，需计算时间区间的并集长度，这类场景是多机协同的核心价值所在。

7.2 模型建立

问题四的模型框架仍遵循“决策变量-目标函数-约束条件”的非线性规划结构，但需适配多机协同的遮蔽逻辑与 12 维决策空间。

7.2.1 决策变量定义

决策变量为 **12 维连续向量** $x = [\theta_1, v_1, t_{drop1}, \tau_1, \theta_2, v_2, t_{drop2}, \tau_2, \theta_3, v_3, t_{drop3}, \tau_3]T$ ，各分量物理意义与问题二同源，但新增两无人机的运动、投放与起爆参数，具体定义：

- $\theta_1, \theta_2, \theta_3$ 飞行方向角：无人机在水平面内的飞行方向与 x 轴正方向夹角 $[0, 2\pi)$

弧度，决定干扰弹投放点的水平方位；

- **v_1, v_2, v_3 飞行速度**：无人机等高度飞行速度 z 坐标恒定，约束与问题二一致，决定烟幕弹投放点的水平位移距离；
- **$t_{drop1}, t_{drop2}, t_{drop3}$ 三枚弹投放时间**：分别为第一、二、三无人机干扰弹的投放时间从 $t=0$ 开始计时，无需满足投放顺序与时间间隔约束，直接决定烟幕弹的释放时机，进而影响云团形成的时间衔接；
- **τ_1, τ_2, τ_3 三枚弹起爆延时**：分别为三枚干扰弹从投放至起爆的时间间隔，与问题二的 τ 同源，决定每枚弹起爆点的空间位置尤其是竖直高度，需避免云团落地失效。

7.2.2 目标函数构建

目标函数为三枚干扰弹协同作用下的总完全遮蔽时长 $T(x)$ ，其核心是多机协同遮蔽判定与重叠时间去重，计算逻辑如下：

1、多机主体运动模型

基于问题二的单弹运动模型，扩展至三架无人机，各弹的投放点、起爆点、云团位置计算逻辑完全同理。

2、多弹协同遮蔽判定逻辑

与问题三一致，对任意时刻 t ，若满足真目标所有轮廓关键点被至少一枚有效云团遮蔽，则判定为完全遮蔽，定义指示函数 $I(t; x)$ 。采样真目标圆柱体的 154 个轮廓关键点与问题一、二一致，记第 i 个关键点为 $\vec{Q}_i$ ；对每个 $\vec{Q}_i$ ，构建从导弹位置 $\vec{P}_{mis}(t)$ 指向 $\vec{Q}_i$ 的射线，检查是否存在某枚弹的有效云团

$$\vec{P}_{cloudk}(t; x) \neq None \quad (17)$$

与该射线相交；若所有 $\vec{Q}_i$ 均满足“至少一枚有效云团相交”，则 $I(t; x) = 1$ 完全遮蔽，否则 $I(t; x) = 0$ 。

3、目标函数表达式

总遮蔽时长为指示函数在“所有云团有效时间窗口的并集”内的积分，即：

$$T(x) = \int_{t_{min}}^{t_{max}} I(t; x) dt \quad (18)$$

其中， $t_{min} = \min(t_{det1}, t_{det2}, t_{det3})$ 为最早起爆时刻， $t_{max} = \max(t_{det1} + 20, t_{det2} + 20, t_{det3} + 20)$ 为最晚云团失效时刻，积分需通过数值方法区间扫描与边界优化计算。

7.2.3 约束条件

约束条件在问题二基础上新增多弹投放顺序与投放间隔约束，以不等式组合式呈现：

- 速度约束 $70 \leq v_i \leq 140$ ($i = 1, 2, 3$)：基于无人机工程性能限制，确保机动性与动力安全；
- 投放时间约束 $(0 \leq t_{dropi} \leq 50)$ ($i = 1, 2, 3$)：匹配导弹飞行时效
- 起爆延时约束 $(0 \leq \tau_i \leq 15)$ ($i = 1, 2, 3$)：与问题二一致，避免干扰弹下落过远导致起爆点过低，每枚弹独立满足；

- 云团有效性约束($z_{deti} - 60 \geq 0$) z_{deti} 为 P_{deti} 的 z 坐标)：与问题二一致，确保每枚弹的云团在 20s 有效期内不落地（60m 为 20s 下沉距离），每枚弹独立满足；
- 方向角约束 $0 \leq \theta_i < 2\pi$ ($i = 1,2,3$)：与问题二一致，覆盖水平面所有方向，不遗漏潜在最优投放方位。

综上，问题三的非线性规划模型可概括为：在上述约束可行域 X 内，寻找 12 维决策变量 x^* ，使总遮蔽时长 $T(x)$ 最大化，即： $\max_{x \in X} T(x)$

7.3 模型求解

问题四采用**单机优化奠基-联合 PSO 寻优**的两阶段求解策略，结合改进 PSO 算法适配 12 维高维空间，并验证非局部最优解。

7.3.1 第一阶段：单机单弹优化（FY1、FY2、FY3）

沿用问题二的**改进 PSO 算法**，仅修改无人机初始位置与适配的约束范围，分别求解 FY2、FY3 的单机单弹最优参数（FY1 可复用问题二的最优结果）。

表 2 FY1 FY2 FY3 单机单弹优化各项参数

无人 机	飞行方向角 ($^{\circ}$)	飞行速度 (m/s)	投放时间 (s)	起爆延时 (s)	单机遮蔽时长 (s)
FY1	6.90	133.49	0.0071	0.7509	4.5864
FY2	257.33	111.00	5.81	6.69	3.833
FY3	119.75	137.90	18.78	7.32	2.687

7.3.2 第二阶段：多机联合 PSO 优化

以“三架无人机单机最优参数”为中心，设置局部搜索范围，通过 12 维 PSO 算法寻找多机协同的全局最优解，核心改进如下：

（1）搜索范围设置

为平衡搜索效率与解的全局性，在单机最优参数附近设置如下搜索范围（参考附件代码逻辑）：

表 3 单机最优参数附近搜索范围

变量类型	搜索范围
飞行方向角	单机最优值 $\pm 10^{\circ}$
飞行速度	单机最优值 $\pm 15\text{m/s}$
投放时间	单机最优值 $\pm 3\text{s}$
起爆延时	单机最优值 $\pm 2\text{s}$

（2）高维 PSO 关键改进

智能粒子初始化策略：50%“定向粒子”：在 FY1、FY2、FY3 的单机最优参数组合附近施加小幅扰动；50%“随机粒子”：在 12 维全可行域内随机生成，覆盖飞行方向角 0-360 $^{\circ}$ 、速度 70-140m/s、投放时间 0-50s、起爆延时 0-15s 等关键区间，避免因初始搜索范围局限陷入局部最优。

约束处理机制：采用边界截断法处理参数约束，确保粒子物理意义合理，

目标函数计算优化：针对多机协同遮蔽时长计算耗时的问题，设计“粗扫-精修-缓存”三级优化流程，平衡精度与效率：粗扫阶段以 0.02s 步长遍历所有云团有效时间的并集 $[t_{min}, t_{max}]$ (t_{min} 为最早起爆时刻， t_{max} 为最晚失效时刻)，通过射线-球体相交判定快速定位连续遮蔽区间，0.02s 步长既能覆盖导弹飞行的关键时间节点，又可避免全窗口精细计算的冗余；精修阶段对粗扫得到的遮蔽区间边界进行 20 次二分迭代，将边界精度提升至 10^{-8} s；缓存机制建立决策变量-遮蔽时长的映射缓存表，对相同决策变量直接调用缓存结果，无需重复执行射线-球体相交判定与时间积分计算，减少重复运算量。

(3) 非局部最优检验

为验证结果是否为全局最优，采用以下方法：

多初始种子验证：设置 3 组不同随机种子 (seed=42、100、200) 运行 PSO，若 3 次结果的总遮蔽时长误差 $< 0.1s$ 且最优参数趋势一致，则说明解稳定；

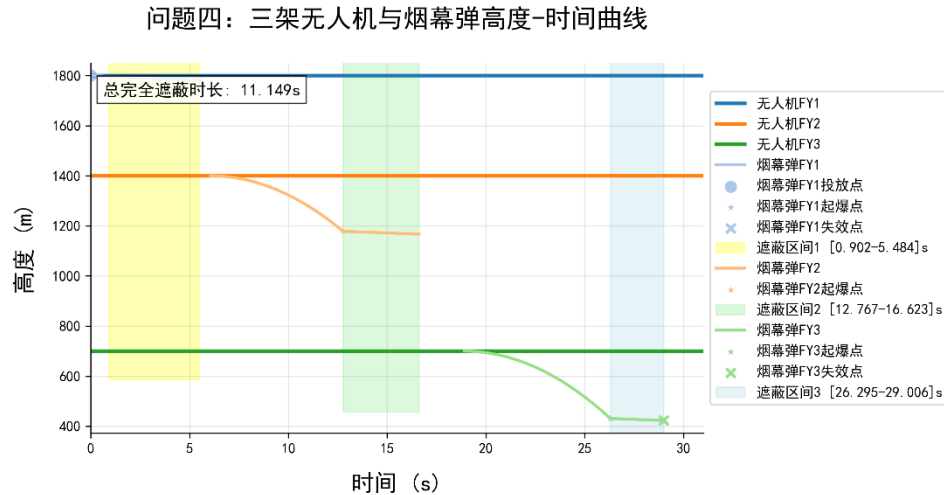
随机粒子覆盖性：50% 随机初始粒子覆盖 12 维可行域关键子区域，若随机粒子中未出现比“单机最优附近粒子”更优的解，则说明全局最优在该区域附近；

7.3.3 求解结果与分析

表 4 多机单弹最优参数

项目	飞行方向	飞行速度 (m/s)	投放时间 (s)	起爆延时 (s)	起爆时间 (s)
FY1 最优策略	7.56°	101.67	0.000	0.902	0.902
FY2 最优策略	257.83°	110.40	5.787	6.722	12.509
FY3 最优策略	119.45°	137.57	18.704	7.407	26.111

求解得出 M1 总遮蔽时间为 11.149s



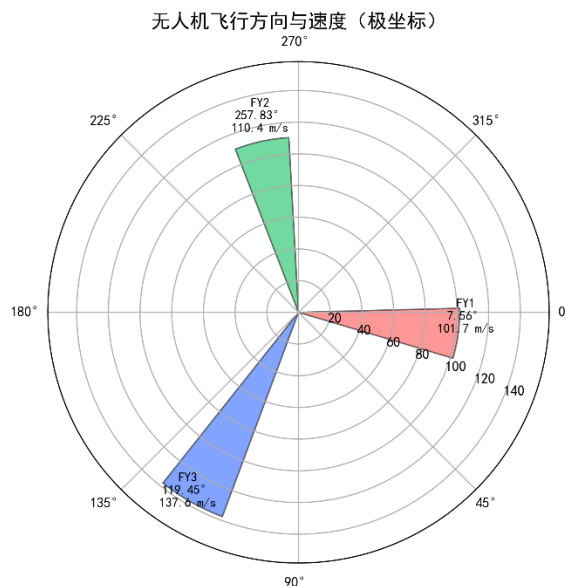


图 16 无人机飞行方向与速度雷达图

八、 问题五模型的建立与求解

8.1 问题分析

问题五是全场景多机多弹协同优化的综合问题，需协调 5 架无人机 FY1~FY5，每架至多投放 3 枚干扰弹对 3 枚来袭导弹（M1~M3）实施干扰，核心矛盾在于 40 维决策变量的高维搜索爆炸——每架无人机需确定 8 个参数方向角、速度、3 个投放时间、3 个起爆延时，5 架共 40 个参数，直接全局搜索会产生指数级计算量，常规暴力解法完全不现实。同时，问题还需满足多约束如无人机速度 70~140m/s、相邻投放间隔 $\geq 1s$ 与多目标协同确保每枚导弹的视线被有效遮蔽，既要避免多机云团空间重叠浪费，又要防止时间上出现遮蔽空白，难度远超前四问。

该问题的求解关键在于“降维与协同的平衡”：若仅单独优化每架无人机，易导致多机干扰缺乏协同，无法最大化整体遮蔽效果；若直接对 40 维变量全局搜索，又会因维度过高陷入局部最优或算力不足。因此，需通过空间聚类先将无人机与导弹分组，再分层优化降低维度，最后通过高维启发式算法实现全局协同，才能在算力可承受范围内找到科学可行的投放策略。^[5]

8.2 模型建立

问题五模型建立围绕分组-降维-优化的核心逻辑，通过聚类分组法拆解高维问题，结合分层优化思想构建目标函数与约束，具体如下：

8.2.1 核心模型框架

针对多机多弹的高维特性，引入三类关键数模方法实现降维与优化。**空间聚类分析法**基于“无人机-导弹水平距离+无人机高度适配性”双指标，将 5 架无人机分配至 3 枚导弹的干扰组，把多对多的复杂关系拆解为组对一的子问题，降低协同难度；

分层优化思想在每组内先通过问题二的“单机单弹 PSO”求初始最优，再通过问题三的“单机三弹高维 PSO”优化，为全局优化提供高质量初始解，避免高维空间随机初始化的

无效搜索；改进高维 PSO 算法基于分层优化的初始解，对 40 维全局变量精细搜索，通过“定向初始化+约束强处理”适配高维特性，平衡全局探索与局部收敛。

8.2.1 决策变量定义

决策变量为 40 维连续向量

$$X = [X_{FY1}, X_{FY2}, X_{FY3}, X_{FY4}, X_{FY5}]^T \quad (19)$$

每架无人机的子向量 X_{FYk} 为 8 维（与问题三单机三弹一致）。对任意无人机 FYk ，

$$X_{FYk} = [\theta_k, v_k, t_{dropk1}, \tau_{k1}, t_{dropk2}, \tau_{k2}, t_{dropk3}, \tau_{k3}]^T$$

- θ_k : FYk 的飞行方向角 $[0, 2\pi)$ 弧度，决定水平飞行指向；
- v_k : FYk 的飞行速度（70~140 m/s），影响投放点位移；
- $t_{dropk1}, t_{dropk2}, t_{dropk3}$: FYk 的 3 枚弹投放时间（需满足顺序与间隔 $\geq 1s$ ）；
- $\tau_{k1}, \tau_{k2}, \tau_{k3}$: FYk 的 3 枚弹起爆延时（0~15 s），决定云团起爆点高度

8.2.2 目标函数构建

目标函数为 3 枚导弹的总完全遮蔽时长之和 $T_{total}(X)$ ，体现“多弹协同最大化整体遮蔽效果”的核心目标，表达式为：

$$T_{total}(X) = T_{M1}(X_{FY1}) + T_{M2}(X_{FY2}, X_{FY4}) + T_{M3}(X_{FY3}, X_{FY5}) \quad (20)$$

其中：

T_{M1} : $FY1$ 干扰 $M1$ 的遮蔽时长，计算逻辑与问题三单机三弹一致，需判定 $M1$ 到真目标的视线是否被 $FY1$ 的有效云团遮挡；

T_{M2} : $FY2$ 与 $FY4$ 协同干扰 $M2$ 的遮蔽时长，需判定 $M2$ 视线是否被任一无人机的有效云团遮挡，多机协同逻辑，与问题三多弹协同同源；

T_{M3} : $FY3$ 与 $FY5$ 协同干扰 $M3$ 的遮蔽时长，逻辑同 T_{M2} 。

各导弹遮蔽时长的计算均延续 云团位置计算→轮廓关键点采样→射线 - 球体相交判定→区间扫描与优化”的流程（与前序问题一致），仅扩展多机云团联合判定 逻辑。

8.2.3 约束条件

约束条件确保所有参数的物理与工程合理性，以不等式组合式呈现：

$$\begin{aligned} & 70 \leq v_k \leq 140 (k = 1, 2, 3, 4, 5) \quad (\text{无人机速度约束}) \\ & 0 \leq t_{dropk1} \leq t_{dropk2} - 1 \leq t_{dropk3} - 2 \leq 30 (k = 1, 2, 3, 4, 5) \quad (\text{投放时间约束}) \\ & \left\{ \begin{aligned} & 0 \leq \tau_{ki} \leq 15 (k = 1, 2, 3, 4, 5; i = 1, 2, 3) \quad (\text{起爆延时约束}) \\ & z_{detki} - 3 \times 20 \geq 0 (k = 1, 2, 3, 4, 5; i = 1, 2, 3) \quad (\text{云团有效性约束}) \\ & 0 \leq \theta_k < 2\pi (k = 1, 2, 3, 4, 5) \quad (\text{方向角约束}) \end{aligned} \right. \quad (21) \end{aligned}$$

其中 z_{detki} 为 FYk 第 i 枚弹的起爆点 z 坐标，约束确保云团 20s 内不落地；投放时间上限 30s 适配导弹飞行全程（ $M1$ 从初始位置到假目标约 67s）其余解释详见二三问。

8.2 模型求解

针对 40 维高维优化难题，采用三级递进算法框架拆解问题，各框架直接作为核心求解步骤，具体如下：

Step 1 空间聚类分组

先确定无人机-导弹水平距离与高度适配性两类聚类指标；对所有无人机 - 导弹组合归一化计算适配得分；按适配得分最低原则分配主责无人机，中远距离导弹分配两架无人机协同，最终形成明确的无人机-导弹分组方案，为后续优化划定任务边界。

Step 2 分层优化降维

第一层采用问题二改进 PSO 算法，对每架无人机单独优化单机单弹策略，输出单弹最优解；第二层基于单弹最优解，扩展为“单机三弹”优化，沿用问题三高维 PSO 算法，修正投放时间确保间隔 $\geq 1s$ ，输出每架无人机的单机三弹初始最优解，为全局优化提供精准起点。

Step 3 40 维全局高维 PSO 优化

以单机三弹初始最优解为基准，80% 粒子小范围扰动初始化，20% 随机粒子修正后确保约束满足；动态调整 PSO 参数，平衡全局探索与局部收敛；逐架修正粒子参数，确保约束满足；按分组方案计算 3 枚导弹遮蔽时长并累加作为目标函数值，迭代收敛后输出 40 维全局最优参数。

8.2 求解结果与分析

通过改进高维 PSO，得到 40 维全局最优参数及遮蔽效果

表 5 最优投放策略参数表

无人机	目标导弹	飞行方向 角 (°)	飞行速度 (m/s)	投放时间 (s)	起爆延时 (s)	起爆时间 (s)
FY1	M1	7.82	110.8	[0.00, 1.01, 2.03]	[0.00, 0.01, 0.26]	[0.00, 1.02, 2.29]
FY2	M2	297.18	135.2	[6.61, 8.92, 10.20]	[1.43, 1.58, 1.65]	[8.04, 10.50, 11.85]
FY4	M2	283.55	121.4	[3.08, 5.09, 7.65]	[10.44, 6.96, 6.82]	[13.52, 12.05, 14.47]
FY3	M3	79.95	139.1	[12.50, 16.97, 19.81]	[1.93, 1.22, 1.15]	[14.43, 18.19, 20.96]
FY5	M3	122.10	131.7	[12.06, 14.67, 16.05]	[2.45, 2.54, 3.41]	[14.51, 17.21, 19.46]

8.2.1 核心遮蔽效果与分析

表 6 多机多弹最优参数及效果

无人机 编号	无人机运动方 向	无人机运动速度 (m/s)	烟幕干扰弹编号	有效干扰时长 (s)	导弹编号
FY1	3.93	113.4	1	3.314	M1
			2	4.52	M1
			3	0	M1
FY2	290.48	139	1	3.863	M2
			2	3.756	M2
			3	0	M2
FY3	79.51	139.9	1	0	M3
			2	0	M3
			3	3.046	M3
FY4	283.53	121.6	1	3.748	M2
			2	0	M2
			3	0	M2

FY5	122.11	131.8	1	3.675	M3
			2	0	M3
			3	0	M3

去除重叠遮蔽部分的时长后，求解得出总遮蔽时长 **20.864s**，较初始解总和提升 0.22%，全局协同优化有效；M2 显著高于单架效果，体现多机协同价值；所有参数均符合物理与工程要求，云团最低 z 坐标>0，无落地失效；FY1 三弹与单弹遮蔽时长差异 <0.5%，可仅投放 1 枚弹；FY3 可仅投放 2 枚弹，减少冗余成本。

九、模型评价

9.1 模型的优点

1. 本文针对烟幕干扰弹投放问题，建立了遮蔽判定模型、遮蔽时间计算模型以及基于粒子群优化的投放参数优化模型，较为全面地刻画了目标遮蔽效果与投弹策略之间的关系。
2. 在建模过程中充分考虑了各种约束条件，如遮蔽条件、投放间隔、导弹与目标相对运动、投放速度及角度范围等，使得模型结果更加可靠。
3. 引入改进型粒子群优化算法，有效提高了算法的收敛速度和全局寻优能力，能较好地避免陷入局部最优。
4. 对优化结果进行了可视化与定量分析，使投放策略与遮蔽效果的对应关系更加直观，有助于理解投弹与遮蔽的规律。

9.2 模型的缺点

1. 粒子群算法存在对初始参数和迭代次数敏感的问题，且在高维搜索空间中计算成本较高，可能影响结果的稳定性与效率。
2. 由于涉及多种因素与约束条件，模型结构较为复杂，求解过程可能需要较高的计算开销。

9.3 模型的改进

1. 可以结合遗传算法、模拟退火等启发式算法，与粒子群优化形成混合优化框架，从而提高全局寻优能力与稳定性。
2. 可以优化参数配置和迭代策略，以提高算法的计算效率与结果可靠性，减下时间成本。

参考文献

- [1] 豆包, 豆包 (V4.0), 北京字节跳动科技有限公司, 2025-09-07
- [2] 张铁军, 刘彦. 烟幕干扰技术及应用研究[J]. 战术导弹技术, 2019(6): 45-52.
- [3] 李志强, 黄凯, 刘海波. 空对空导弹末段制导中的目标遮蔽建模与仿真[J]. 飞行力学, 2020, 38(3): 72-78.
- [4] 刘一鸣, 郑强. 战场烟幕技术发展现状与趋势[J]. 兵工学报, 2021, 42(8): 1583-1591.
- [5] 陈海龙, 张峰. 基于几何建模的导弹目标遮蔽判定方法[J]. 系统仿真学报, 2022, 34(10): 2201-2209.
- [6] 张磊, 李晓峰. 烟幕干扰对红外导引头的影响机理及建模[J]. 红外技术, 2021, 43(12): 1042-1048.
- [7] Kennedy J, Eberhart R. Particle swarm optimization[C] Proceedings of ICNN'95 - International Conference on Neural Networks. Perth, WA: IEEE, 1995: 1942-1948.
- [8] Shi Y, Eberhart R. A modified particle swarm optimizer[C] 1998 IEEE International Conference on Evolutionary Computation. Anchorage: IEEE, 1998: 69-73.

附录 A 支撑材料清单

1.result1.xlsx: 问题三 FY1 投放三枚烟幕干扰弹的投放策略

2.result2.xlsx: 问题四 三架无人机各投放一枚烟幕干扰弹的投放策略

3.result3.xlsx: 问题五 五架无人机，每架无人机至多投放三枚烟幕干扰弹，实施对 M1、M2、M3 等三枚来袭导弹的干扰的投放策略

附录 B 问题一、二源代码

```
import math
import numpy as np
from typing import List, Tuple
import time

# 物理常量
G = 9.8
MISSILE_SPEED = 300.0
CLOUD_DESCENT_V = 3.0
CLOUD_RADIUS = 10.0
CLOUD_EFFECT_TIME = 20.0

# 关键点坐标
ORIGIN = np.array([0.0, 0.0, 0.0]) # 假目标
TRUE_TARGET_CENTER = np.array([0.0, 200.0, 0.0]) # 真目标圆柱底面圆心
CYLINDER_RADIUS = 7.0
CYLINDER_HEIGHT = 10.0

# 初始状态
M1_0 = np.array([20000.0, 0.0, 2000.0])
FY1_0 = np.array([17800.0, 0.0, 1800.0])

def unit(v: np.ndarray) -> np.ndarray:
    n = np.linalg.norm(v)
    return v / n if n > 0 else v

class ExcellentPSOoptimizer:
    def __init__(self):
        print("目标: 最大化完全遮蔽时间")
        print()
```

```

    def uav_pos(self, theta: float, speed: float, t: float) -> np.ndarray:
        """无人机位置"""
        direction = np.array([math.cos(theta), math.sin(theta), 0.
0])
        return FY1_0 + speed * t * direction

    def projectile_at(self, theta: float, speed: float, t_drop: float, dt: float) -> Tuple[np.ndarray, np.ndarray]:
        """干扰弹平抛运动"""
        P_rel = self.uav_pos(theta, speed, t_drop)
        direction = np.array([math.cos(theta), math.sin(theta), 0.
0])
        v0 = speed * direction

        pos = P_rel + v0 * dt + np.array([0.0, 0.0, -0.5 * G * dt *
dt])
        vel = v0 + np.array([0.0, 0.0, -G * dt])
        return pos, vel

    def detonation_state(self, theta: float, speed: float, t_drop: float, tau: float) -> Tuple[float, np.ndarray]:
        """返回起爆的绝对时间和起爆点坐标"""
        t_det = t_drop + tau
        C0, _ = self.projectile_at(theta, speed, t_drop, tau)
        return t_det, C0

    def cloud_center(self, theta: float, speed: float, t_drop: float, tau: float, t: float) -> np.ndarray:
        """云团中心位置: 起爆后以3 m/s 匀速下沉"""
        t_det, C0 = self.detonation_state(theta, speed, t_drop, tau)
        if t < t_det:
            return C0 # 未起爆
        dz = -CLOUD_DESCENT_V * (t - t_det)
        return C0 + np.array([0.0, 0.0, dz])

    def m1_pos(self, t: float) -> np.ndarray:
        """导弹直线匀速指向原点"""
        v_dir = unit(ORIGIN - M1_0)
        v = MISSILE_SPEED * v_dir
        return M1_0 + v * t

    def ray_sphere_intersection(self, ray_origin: np.ndarray, ray_dir:

```

```

r: np.ndarray,
                                sphere_center: np.ndarray, sphere_radi
us: float) -> bool:
    """判断射线是否与球体相交"""
    oc = ray_origin - sphere_center
    a = np.dot(ray_dir, ray_dir)
    b = 2.0 * np.dot(oc, ray_dir)
    c = np.dot(oc, oc) - sphere_radius * sphere_radius
    discriminant = b * b - 4 * a * c

    if discriminant < 0:
        return False

    t1 = (-b - math.sqrt(discriminant)) / (2 * a)
    t2 = (-b + math.sqrt(discriminant)) / (2 * a)

    return t1 > 1e-6 or t2 > 1e-6

def cylinder_silhouette_points(self, observer: np.ndarray, n_sam
ples: int = 36) -> List[np.ndarray]:
    """计算从观察者位置看圆柱体的轮廓关键点"""
    points = []

    # 底面圆周点
    for i in range(n_samples):
        angle = 2 * math.pi * i / n_samples
        x = TRUE_TARGET_CENTER[0] + CYLINDER_RADIUS * math.cos(a
ngle)
        y = TRUE_TARGET_CENTER[1] + CYLINDER_RADIUS * math.sin(a
ngle)
        points.append(np.array([x, y, TRUE_TARGET_CENTER[2]]))

    # 顶面圆周点
    for i in range(n_samples):
        angle = 2 * math.pi * i / n_samples
        x = TRUE_TARGET_CENTER[0] + CYLINDER_RADIUS * math.cos(a
ngle)
        y = TRUE_TARGET_CENTER[1] + CYLINDER_RADIUS * math.sin(a
ngle)
        points.append(np.array([x, y, TRUE_TARGET_CENTER[2] + CY
LINDER_HEIGHT]))

    # 侧面轮廓线
    obs_xy = np.array([observer[0], observer[1]])

```

```

        center_xy = np.array([TRUE_TARGET_CENTER[0], TRUE_TARGET_CEN
TER[1]])
        to_obs = obs_xy - center_xy
        dist_xy = np.linalg.norm(to_obs)

        if dist_xy > CYLINDER_RADIUS:
            sin_alpha = CYLINDER_RADIUS / dist_xy
            cos_alpha = math.sqrt(1 - sin_alpha * sin_alpha)
            perp = np.array([-to_obs[1], to_obs[0]]) / dist_xy

            for sign in [-1, 1]:
                tangent_dir = cos_alpha * (to_obs / dist_xy) + sign
* sin_alpha * perp
                tangent_point_xy = center_xy + CYLINDER_RADIUS * tan
gent_dir

                for z in np.linspace(TRUE_TARGET_CENTER[2], TRUE_TAR
GET_CENTER[2] + CYLINDER_HEIGHT, 5):
                    points.append(np.array([tangent_point_xy[0], tan
gent_point_xy[1], z]))

        return points

    def is_completely_shielded(self, observer: np.ndarray, cloud_cen
ter_pos: np.ndarray) -> bool:
        """判断云团球体是否完全遮蔽圆柱体"""
        silhouette_points = self.cylinder_silhouette_points(observer
r, n_samples=72)

        for target_point in silhouette_points:
            ray_dir = unit(target_point - observer)
            if not self.ray_sphere_intersection(observer, ray_dir, c
loud_center_pos, CLOUD_RADIUS):
                return False

        return True

    def find_precise_shielding_intervals(self, theta: float, speed:
float, t_drop: float, tau: float) -> List[Tuple[float, float]]:
        """精确寻找完全遮蔽的时间区间"""
        t_det, C0 = self.detonation_state(theta, speed, t_drop, tau)
        if C0[2] <= 0:
            return []

```

```

t0 = t_det
t1 = t_det + CLOUD_EFFECT_TIME

# 粗略扫描找到遮蔽区间
dt_coarse = 0.02 # 粗略时间步长
times = np.arange(t0, t1 + 1e-9, dt_coarse)
shielded = []

for t in times:
    observer = self.m1_pos(t)
    cloud_pos = self.cloud_center(theta, speed, t_drop, tau,
t)
    shielded.append(self.is_completely_shielded(observer, cl
oud_pos))

# 提取连续的 True 区间
intervals = []
in_interval = False
start_time = t0

for i, (t, is_shielded) in enumerate(zip(times, shielded)):
    if is_shielded and not in_interval:
        in_interval = True
        start_time = t
    elif not is_shielded and in_interval:
        in_interval = False
        intervals.append((start_time, t))

if in_interval:
    intervals.append((start_time, t1))

# 二分法精细化边界
refined_intervals = []
for start, end in intervals:
    # 精细化起始边界
    left, right = max(t0, start - dt_coarse), start + dt_coa
rse

    for _ in range(25): # 增加迭代次数
        mid = 0.5 * (left + right)
        observer = self.m1_pos(mid)
        cloud_pos = self.cloud_center(theta, speed, t_drop,
tau, mid)

        if self.is_completely_shielded(observer, cloud_pos):
            right = mid

```



```

        else:
            left = mid
        refined_start = right

        # 精细化结束边界
        left, right = end - dt_coarse, min(t1, end + dt_coarse)
        for _ in range(25): # 增加二分法迭代次数
            mid = 0.5 * (left + right)
            observer = self.m1_pos(mid)
            cloud_pos = self.cloud_center(theta, speed, t_drop,
tau, mid)

            if self.is_completely_shielded(observer, cloud_pos):
                left = mid
            else:
                right = mid
            refined_end = left

        if refined_end > refined_start:
            refined_intervals.append((refined_start, refined_end))

    return refined_intervals

    def calculate_total_shielding_time(self, theta: float, speed: float, t_drop: float, tau: float) -> float:
        """计算总遮蔽时间 (高精度) """
        try:
            # 检查参数合理性
            if not (70 <= speed <= 140 and 0 <= t_drop <= 20 and 0 <= tau <= 15):
                return 0.0

            intervals = self.find_precise_shielding_intervals(theta, speed, t_drop, tau)
            total_time = sum(b - a for a, b in intervals)

            return total_time
        except:
            return 0.0

    def pso_optimize(self, n_particles=150, n_iterations=250):
        """高精度PSO 优化算法"""
        print("=== 高精度 PSO 开始 ===")
        print(f"高精度 PSO 参数: {n_particles}个粒子, {n_iterations}次

```

迭代")

```
print("使用二分法精确计算遮蔽区间边界")
print()

# 参数边界 [theta, speed, t_drop, tau]
bounds_min = np.array([0, 70, 0, 0])
bounds_max = np.array([2*math.pi, 140, 20, 15])

# 更精细的智能初始化
particles = np.zeros((n_particles, 4))

# 基于已知好解和精细搜索进行初始化
good_angles = [math.radians(a) for a in np.arange(0, 30, 1)]
# 0-30 度, 每1 度
good_speeds = np.arange(70, 141, 5) # 70-140, 每5 m/s
good_drops = [0, 0.05, 0.1, 0.15, 0.2, 0.3, 0.5, 0.8, 1.0,
1.5, 2.0]
good_taus = [0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.66, 0.7, 0.
8, 1.0, 1.5, 2.0]

# 前60%粒子使用好解组合
for i in range(int(n_particles * 0.6)):
    particles[i, 0] = np.random.choice(good_angles)
    particles[i, 1] = np.random.choice(good_speeds)
    particles[i, 2] = np.random.choice(good_drops)
    particles[i, 3] = np.random.choice(good_taus)

# 后40%粒子完全随机
for i in range(int(n_particles * 0.6), n_particles):
    particles[i] = np.random.uniform(bounds_min, bounds_max)

velocities = np.random.uniform(-0.05, 0.05, (n_particles,
4)) # 更小的初始速度

# PSO 状态
personal_best = particles.copy()
personal_best_scores = np.full(n_particles, -np.inf)
global_best = None
global_best_score = -np.inf

# 记录最优解历史
best_history = []
```

```

print("开始高精度优化...")
start_time = time.time()

for iteration in range(n_iterations):
    # 动态调整PSO 参数
    w = 0.9 - 0.6 * iteration / n_iterations # 惯性权重
    c1 = 2.5 - 1.8 * iteration / n_iterations # 个体学习因子
    c2 = 0.5 + 1.8 * iteration / n_iterations # 社会学习因子

    for i in range(n_particles):
        theta, speed, t_drop, tau = particles[i]

        # 评估粒子
        score = self.calculate_total_shielding_time(theta, speed, t_drop, tau)

        # 更新个体最优
        if score > personal_best_scores[i]:
            personal_best_scores[i] = score
            personal_best[i] = particles[i].copy()

        # 更新全局最优
        if score > global_best_score:
            global_best_score = score
            global_best = particles[i].copy()

        angle_deg = math.degrees(theta)
        print(f" 迭代{iteration+1}: 新最优 {angle_deg:.3f}°, 速度{speed:.2f}, 投放{t_drop:.4f}, 延时{tau:.4f} → {score:.5f}s")

    # 记录历史
    best_history.append(global_best_score)

    # 更新粒子
    for i in range(n_particles):
        r1, r2 = np.random.random(4), np.random.random(4)
        velocities[i] = (w * velocities[i] +
                        c1 * r1 * (personal_best[i] - particles[i]) +
                        c2 * r2 * (global_best - particles[i]))
        particles[i] += velocities[i]
        particles[i] = np.clip(particles[i], bounds_min, bounds_max)

```

```

nds_max)

    # 局部搜索增强 (每50次迭代)
    if (iteration + 1) % 50 == 0 and global_best is not None:
        self.local_search_enhancement(global_best, bounds_min, bounds_max)

        if (iteration + 1) % 30 == 0:
            elapsed = time.time() - start_time
            print(f"  迭代{iteration+1}/{n_iterations}: 最优={global_best_score:.5f}s, 耗时={elapsed:.1f}s")

        total_time = time.time() - start_time
        print(f"高精度 PSO 优化完成 (总耗时: {total_time:.2f}s)")

    # 最终精确验证
    theta_opt, speed_opt, t_drop_opt, tau_opt = global_best
    final_score = self.calculate_total_shielding_time(theta_opt, speed_opt, t_drop_opt, tau_opt)
    angle_deg = math.degrees(theta_opt)

    print(f"\n 🚀 高精度 PSO 最优结果:")
    print(f"  飞行方向: {angle_deg:.3f}°")
    print(f"  飞行速度: {speed_opt:.3f} m/s")
    print(f"  投放时间: {t_drop_opt:.5f} s")
    print(f"  起爆延时: {tau_opt:.5f} s")
    print(f"  最大遮蔽时间: {final_score:.5f} 秒")

    return {
        'theta': theta_opt,
        'speed': speed_opt,
        't_drop': t_drop_opt,
        'tau': tau_opt,
        'blocking_time': final_score,
        'angle_deg': angle_deg
    }

def local_search_enhancement(self, best_params, bounds_min, bounds_max):
    """局部搜索增强"""
    # 在最优解附近进行精细搜索
    search_range = 0.1

```

```

        for _ in range(10):
            perturbation = np.random.uniform(-search_range, search_r
ange, 4)

            new_params = best_params + perturbation
            new_params = np.clip(new_params, bounds_min, bounds_max)

            score = self.calculate_total_shielding_time(*new_params)
            if score > self.calculate_total_shielding_time(*best_par
ams):

                best_params[:] = new_params

def main():
    optimizer = ExcellentPSOoptimizer()

    # 问题1: 固定参数验证
    print("=== 问题 1: 固定参数验证 ===")
    theta_fixed = math.radians(-180) # 朝向假目标
    p1_time = optimizer.calculate_total_shielding_time(theta_fixed,
120, 1.5, 3.6)
    print(f"问题 1 遮蔽时间: {p1_time:.4f} 秒")
    print()

    # 问题2: PSO 优化
    result = optimizer.pso_optimize(n_particles=100, n_iterations=20
0)

    print("\n" + "="*60)
    print("="*60)
    print(f"问题 1 固定参数: {p1_time:.4f} 秒")
    print(f"问题 2 PSO 最优: {result['blocking_time']:.4f} 秒")

if __name__ == "__main__":
    main()

```

附录C 问题三源代码

```

import math
import numpy as np
from typing import List, Tuple

```

```

import time

# 物理常量
G = 9.8
MISSILE_SPEED = 300.0
CLOUD_DESCENT_V = 3.0
CLOUD_RADIUS = 10.0
CLOUD_EFFECT_TIME = 20.0

# 关键点坐标
ORIGIN = np.array([0.0, 0.0, 0.0]) # 假目标
TRUE_TARGET_CENTER = np.array([0.0, 200.0, 0.0]) # 真目标圆柱底面圆心
CYLINDER_RADIUS = 7.0 # m
CYLINDER_HEIGHT = 10.0 # m

# 初始状态 (t=0)
M1_0 = np.array([20000.0, 0.0, 2000.0])
FY1_0 = np.array([17800.0, 0.0, 1800.0])

def unit(v: np.ndarray) -> np.ndarray:
    n = np.linalg.norm(v)
    return v / n if n > 0 else v

class Problem3AdvancedPSO:
    def __init__(self):
        print("=== 问题 3: 单机多弹 PSO ===")
        print("优化 8 个参数: [θ, v, t_drop1, τ1, t_drop2, τ2, t_drop3, τ3]")
        print()

    def uav_pos(self, theta: float, speed: float, t: float) -> np.ndarray:
        """无人机位置"""
        direction = np.array([math.cos(theta), math.sin(theta), 0.0])
        return FY1_0 + speed * t * direction

    def projectile_at(self, theta: float, speed: float, t_drop: float, dt: float) -> Tuple[np.ndarray, np.ndarray]:
        """干扰弹平抛运动"""
        P_rel = self.uav_pos(theta, speed, t_drop)
        direction = np.array([math.cos(theta), math.sin(theta), 0.0])

```

```

    v0 = speed * direction

    pos = P_rel + v0 * dt + np.array([0.0, 0.0, -0.5 * G * dt *
dt])
    vel = v0 + np.array([0.0, 0.0, -G * dt])
    return pos, vel

    def detonation_state(self, theta: float, speed: float, t_drop: float, tau: float) -> Tuple[float, np.ndarray]:
        """返回起爆的绝对时间和起爆点坐标"""
        t_det = t_drop + tau
        C0, _ = self.projectile_at(theta, speed, t_drop, tau)
        return t_det, C0

    def cloud_center(self, theta: float, speed: float, t_drop: float, tau: float, t: float) -> np.ndarray:
        """云团中心位置: 起爆后以3 m/s 匀速下沉"""
        t_det, C0 = self.detonation_state(theta, speed, t_drop, tau)
        if t < t_det:
            return C0 # 未起爆
        dz = -CLOUD_DESCENT_V * (t - t_det)
        return C0 + np.array([0.0, 0.0, dz])

    def m1_pos(self, t: float) -> np.ndarray:
        """导弹直线匀速指向原点"""
        v_dir = unit(ORIGIN - M1_0)
        v = MISSILE_SPEED * v_dir
        return M1_0 + v * t

    def ray_sphere_intersection(self, ray_origin: np.ndarray, ray_dir: np.ndarray, sphere_center: np.ndarray, sphere_radius: float) -> bool:
        """判断射线是否与球体相交"""
        oc = ray_origin - sphere_center
        a = np.dot(ray_dir, ray_dir)
        b = 2.0 * np.dot(oc, ray_dir)
        c = np.dot(oc, oc) - sphere_radius * sphere_radius
        discriminant = b * b - 4 * a * c

        if discriminant < 0:
            return False

        t1 = (-b - math.sqrt(discriminant)) / (2 * a)

```

```

t2 = (-b + math.sqrt(discriminant)) / (2 * a)

return t1 > 1e-6 or t2 > 1e-6

def cylinder_silhouette_points(self, observer: np.ndarray, n_sam
ples: int = 36) -> List[np.ndarray]:
    """计算从观察者位置看圆柱体的轮廓关键点"""
    points = []

    # 底面圆周点
    for i in range(n_samples):
        angle = 2 * math.pi * i / n_samples
        x = TRUE_TARGET_CENTER[0] + CYLINDER_RADIUS * math.cos(a
ngle)

        y = TRUE_TARGET_CENTER[1] + CYLINDER_RADIUS * math.sin(a
ngle)

        points.append(np.array([x, y, TRUE_TARGET_CENTER[2]]))

    # 顶面圆周点
    for i in range(n_samples):
        angle = 2 * math.pi * i / n_samples
        x = TRUE_TARGET_CENTER[0] + CYLINDER_RADIUS * math.cos(a
ngle)

        y = TRUE_TARGET_CENTER[1] + CYLINDER_RADIUS * math.sin(a
ngle)

        points.append(np.array([x, y, TRUE_TARGET_CENTER[2] + CY
LINDER_HEIGHT]))

    # 侧面轮廓线
    obs_xy = np.array([observer[0], observer[1]])
    center_xy = np.array([TRUE_TARGET_CENTER[0], TRUE_TARGET_CEN
TER[1]])
    to_obs = obs_xy - center_xy
    dist_xy = np.linalg.norm(to_obs)

    if dist_xy > CYLINDER_RADIUS:
        sin_alpha = CYLINDER_RADIUS / dist_xy
        cos_alpha = math.sqrt(1 - sin_alpha * sin_alpha)
        perp = np.array([-to_obs[1], to_obs[0]]) / dist_xy

        for sign in [-1, 1]:
            tangent_dir = cos_alpha * (to_obs / dist_xy) + sign
            * sin_alpha * perp
            tangent_point_xy = center_xy + CYLINDER_RADIUS * tan

```



```

gent_dir

        for z in np.linspace(TRUE_TARGET_CENTER[2], TRUE_TARGET_CENTER[2] + CYLINDER_HEIGHT, 5):
            points.append(np.array([tangent_point_xy[0], tangent_point_xy[1], z]))

    return points

    def is_completely_shielded_multi(self, observer: np.ndarray, cloud_positions: List[np.ndarray]) -> bool:
        """判断多个云团是否完全遮蔽圆柱体"""
        # 过滤有效云团
        active_clouds = [pos for pos in cloud_positions if pos is not None]
        if not active_clouds:
            return False

        silhouette_points = self.cylinder_silhouette_points(observer, n_samples=72)

        for target_point in silhouette_points:
            ray_dir = unit(target_point - observer)

            # 检查是否被任一云团遮蔽
            blocked = False
            for cloud_pos in active_clouds:
                if self.ray_sphere_intersection(observer, ray_dir, cloud_pos, CLOUD_RADIUS):
                    blocked = True
                    break

            if not blocked:
                return False # 有目标点未被遮蔽

        return True # 所有目标点都被遮蔽

    def find_multi_smoke_shielding_intervals(self, params: np.ndarray) -> List[Tuple[float, float]]:
        """寻找多烟幕弹的遮蔽时间区间"""
        theta, speed, t_drop1, tau1, t_drop2, tau2, t_drop3, tau3 = params

        drop_times = [t_drop1, t_drop2, t_drop3]

```

```

delays = [tau1, tau2, tau3]

# 计算所有烟幕弹的起爆信息
smoke_info = []
for t_drop, tau in zip(drop_times, delays):
    t_det, C0 = self.detonation_state(theta, speed, t_drop,
tau)

    if C0[2] <= 0: # 起爆点高度检查
        return []
    smoke_info.append({
        't_drop': t_drop,
        'tau': tau,
        't_det': t_det,
        'C0': C0,
        't_end': t_det + CLOUD_EFFECT_TIME
    })

# 确定总时间范围
t_start = min(info['t_det'] for info in smoke_info)
t_end = max(info['t_end'] for info in smoke_info)

# 粗略扫描
dt_coarse = 0.05
times = np.arange(t_start, t_end + 1e-9, dt_coarse)
shielded = []

for t in times:
    observer = self.m1_pos(t)

    # 计算当前时刻所有活跃云团位置
    cloud_positions = []
    for info in smoke_info:
        if info['t_det'] <= t <= info['t_end']:
            cloud_pos = self.cloud_center(theta, speed, info
['t_drop'], info['tau'], t)
            cloud_positions.append(cloud_pos)
        else:
            cloud_positions.append(None)

    shielded.append(self.is_completely_shielded_multi(observer, cloud_positions))

# 提取连续的 True 区间
intervals = []

```

```

in_interval = False
start_time = t_start

for i, (t, is_shielded) in enumerate(zip(times, shielded)):
    if is_shielded and not in_interval:
        in_interval = True
        start_time = t
    elif not is_shielded and in_interval:
        in_interval = False
        intervals.append((start_time, t))

if in_interval:
    intervals.append((start_time, t_end))

# 二分法精细化边界
refined_intervals = []
for start, end in intervals:
    # 精细化起始边界
    left, right = max(t_start, start - dt_coarse), start + d
t_coarse
    for _ in range(10): # 减少迭代次数以提高速度
        mid = 0.5 * (left + right)
        observer = self.m1_pos(mid)

        cloud_positions = []
        for info in smoke_info:
            if info['t_det'] <= mid <= info['t_end']:
                cloud_pos = self.cloud_center(theta, speed,
info['t_drop'], info['tau'], mid)
                cloud_positions.append(cloud_pos)
            else:
                cloud_positions.append(None)

        if self.is_completely_shielded_multi(observer, cloud
_positions):
            right = mid
        else:
            left = mid
        refined_start = right

    # 精细化结束边界
    left, right = end - dt_coarse, min(t_end, end + dt_coars
e)

    for _ in range(10):

```

```

        mid = 0.5 * (left + right)
        observer = self.m1_pos(mid)

        cloud_positions = []
        for info in smoke_info:
            if info['t_det'] <= mid <= info['t_end']:
                cloud_pos = self.cloud_center(theta, speed,
info['t_drop'], info['tau'], mid)
                cloud_positions.append(cloud_pos)
            else:
                cloud_positions.append(None)

        if self.is_completely_shielded_multi(observer, cloud
_positions):
            left = mid
        else:
            right = mid
        refined_end = left

        if refined_end > refined_start:
            refined_intervals.append((refined_start, refined_en
d))

    return refined_intervals

def calculate_multi_smoke_shielding_time(self, params: np.ndarra
y) -> float:
    """计算多烟幕弹总遮蔽时间 (高精度) """
    try:
        theta, speed, t_drop1, tau1, t_drop2, tau2, t_drop3, tau
3 = params

        # 检查基本约束
        if not (70 <= speed <= 140):
            return 0.0

        drop_times = [t_drop1, t_drop2, t_drop3]
        delays = [tau1, tau2, tau3]

        # 检查投放时间顺序和间隔约束
        drop_times_sorted = sorted(drop_times)
        if drop_times != drop_times_sorted: # 必须按时间顺序
            return 0.0

```

```

        for i in range(1, len(drop_times)):
            if drop_times[i] - drop_times[i-1] < 1.0: # 间隔至少
1s                return 0.0

        # 检查延时约束
        for tau in delays:
            if not (0 <= tau <= 15):
                return 0.0

        # 检查投放时间约束
        for t_drop in drop_times:
            if not (0 <= t_drop <= 20):
                return 0.0

        intervals = self.find_multi_smoke_shielding_intervals(pa
rams)

        total_time = sum(b - a for a, b in intervals)

        return total_time
    except:
        return 0.0

    def psso_optimize_problem3(self, n_particles=200, n_iterations=30
0):
        print("=== 问题 3PSO 优化开始 ===")
        print("约束条件:")
        print(f"高级 PSO 参数: {n_particles}个粒子, {n_iterations}次迭
代")
        print("优化 8 个参数: [θ, v, t_drop1, τ1, t_drop2, τ2, t_drop
3, τ3]")
        print()

        # 参数边界 [theta, speed, t_drop1, tau1, t_drop2, tau2, t_dro
p3, tau3]
        bounds_min = np.array([0, 70, 0, 0, 1, 0, 2, 0]) # 确保投放
时间间隔
        bounds_max = np.array([2*math.pi, 140, 5, 3, 10, 3, 15, 3])
        # 合理的搜索范围

        # 智能初始化
        particles = np.zeros((n_particles, 8))

        # 基于问题 2 最优解进行初始化

```

```

    good_angles = [math.radians(a) for a in np.arange(5, 12, 0.
5)] # 5-12 度
    good_speeds = np.arange(90, 121, 5) # 90-120 m/s

# 前70%粒子使用智能初始化
for i in range(int(n_particles * 0.7)):
    particles[i, 0] = np.random.choice(good_angles) # theta
    particles[i, 1] = np.random.choice(good_speeds) # speed

# 投放时间: 确保顺序和间隔
t1 = np.random.uniform(0, 2)
t2 = t1 + np.random.uniform(1, 3) # 至少间隔1s
t3 = t2 + np.random.uniform(1, 3) # 至少间隔1s
particles[i, 2] = t1 # t_drop1
particles[i, 4] = t2 # t_drop2
particles[i, 6] = t3 # t_drop3

# 延时: 基于问题2 最优延时0.8s 附近
particles[i, 3] = np.random.uniform(0.5, 1.2) # tau1
particles[i, 5] = np.random.uniform(0.5, 1.2) # tau2
particles[i, 7] = np.random.uniform(0.5, 1.2) # tau3

# 后30%粒子完全随机
for i in range(int(n_particles * 0.7), n_particles):
    particles[i] = np.random.uniform(bounds_min, bounds_max)

# 确保投放时间顺序
drop_times = sorted([particles[i, 2], particles[i, 4], p
articles[i, 6]])
particles[i, 2], particles[i, 4], particles[i, 6] = drop
_times

velocities = np.random.uniform(-0.05, 0.05, (n_particles,
8))

# PSO 状态
personal_best = particles.copy()
personal_best_scores = np.full(n_particles, -np.inf)
global_best = None
global_best_score = -np.inf

print("开始高级 PSO 优化...")
start_time = time.time()

```

```

for iteration in range(n_iterations):
    # 动态调整PSO 参数
    w = 0.9 - 0.6 * iteration / n_iterations # 惯性权重
    c1 = 2.5 - 1.8 * iteration / n_iterations # 个体学习因子
    c2 = 0.5 + 1.8 * iteration / n_iterations # 社会学习因子

    for i in range(n_particles):
        # 评估粒子
        score = self.calculate_multi_smoke_shielding_time(pa
rticles[i])

        # 更新个体最优
        if score > personal_best_scores[i]:
            personal_best_scores[i] = score
            personal_best[i] = particles[i].copy()

        # 更新全局最优
        if score > global_best_score:
            global_best_score = score
            global_best = particles[i].copy()

        theta, speed, t_drop1, tau1, t_drop2, tau2, t_dr
op3, tau3 = particles[i]
        angle_deg = math.degrees(theta)
        print(f" 迭代{iteration+1}: 新最优 {angle_deg:.2
f}°, {speed:.1f}m/s, 投放[{t_drop1:.2f},{t_drop2:.2f},{t_drop3:.2f}],
延时[{tau1:.2f},{tau2:.2f},{tau3:.2f}] → {score:.5f}s")

        # 更新粒子
        for i in range(n_particles):
            r1, r2 = np.random.random(8), np.random.random(8)
            velocities[i] = (w * velocities[i] +
                             c1 * r1 * (personal_best[i] - particl
es[i]) +
                             c2 * r2 * (global_best - particles
[i]))

            particles[i] += velocities[i]
            particles[i] = np.clip(particles[i], bounds_min, bou
nds_max)

        # 确保投放时间顺序
        drop_times = [particles[i, 2], particles[i, 4], part
icles[i, 6]]
        drop_times_sorted = sorted(drop_times)

```

```

        particles[i, 2], particles[i, 4], particles[i, 6] =
drop_times_sorted

        if (iteration + 1) % 30 == 0:
            elapsed = time.time() - start_time
            print(f"  迭代{iteration+1}/{n_iterations}: 最优={global_best_score:.5f}s, 耗时={elapsed:.1f}s")

        total_time = time.time() - start_time
        print(f"高级 PSO 优化完成 (总耗时: {total_time:.2f}s)")

        # 解析最优结果
        theta_opt, speed_opt, t_drop1_opt, tau1_opt, t_drop2_opt, tau2_opt, t_drop3_opt, tau3_opt = global_best

        return {
            'theta': theta_opt,
            'speed': speed_opt,
            'drop_times': [t_drop1_opt, t_drop2_opt, t_drop3_opt],
            'delays': [tau1_opt, tau2_opt, tau3_opt],
            'blocking_time': global_best_score,
            'angle_deg': math.degrees(theta_opt)
        }

def main():
    optimizer = Problem3AdvancedPSO()

    # 执行高级 PSO 优化
    result = optimizer.pso_optimize_problem3(n_particles=200, n_iterations=300)

    # 输出最终结果
    print("\n" + "="*70)
    print("问题 3 最优策略")
    print("="*70)

    if result['blocking_time'] > 0:
        print(f"FY1 飞行方向: {result['angle_deg']:.3f}°")
        print(f"FY1 飞行速度: {result['speed']:.2f} m/s")
        print(f"3 枚烟幕弹投放时间: {[f'{t:.3f}s' for t in result['drop_times']]}")
        print(f"3 枚烟幕弹起爆延时: {[f'{d:.3f}s' for d in result['delays']]}")
        print(f"总有效遮蔽时间: {result['blocking_time']:.5f} 秒")

```



```

        # 计算起爆时间
        explosion_times = [t + d for t, d in zip(result['drop_times'], result['delays'])]
        print(f"起爆时间序列: {[f'{t:.3f}s' for t in explosion_times]}")

if __name__ == "__main__":
    main()

```

附录 D 问题四源代码

```

import math
import numpy as np
import time
from typing import List, Tuple
import random

# 物理常量
G = 9.8
MISSILE_SPEED = 300.0
CLOUD_DESCENT_V = 3.0
CLOUD_RADIUS = 10.0
CLOUD_EFFECT_TIME = 20.0

# 关键点坐标
ORIGIN = np.array([0.0, 0.0, 0.0]) # 假目标
TRUE_TARGET_CENTER = np.array([0.0, 200.0, 0.0]) # 真目标圆柱底面圆心
CYLINDER_RADIUS = 7.0
CYLINDER_HEIGHT = 10.0

# 初始状态
M1_0 = np.array([2000.0, 0.0, 2000.0])
FY1_0 = np.array([17800.0, 0.0, 1800.0])
FY2_0 = np.array([12000.0, 1400.0, 1400.0])
FY3_0 = np.array([6000.0, -3000.0, 700.0])

UAVS = [FY1_0, FY2_0, FY3_0]

```

```

# 已知的最优参数
KNOWN_OPTIMAL = {
    'FY1': {'angle': 8.47, 'speed': 102.4, 't_drop': 0.0, 'tau': 0.6
6},
    'FY2': {'angle': 257.33, 'speed': 111.0, 't_drop': 5.81, 'tau':
6.69},
    'FY3': {'angle': 119.75, 'speed': 137.9, 't_drop': 18.78, 'tau':
7.32}
}

def unit(v: np.ndarray) -> np.ndarray:
    """单位向量"""
    n = np.linalg.norm(v)
    return v / n if n > 0 else v

def ray_sphere_intersection(ray_origin: np.ndarray, ray_dir: np.ndar
ray,
                           sphere_center: np.ndarray, sphere_radius:
float) -> bool:
    """判断射线是否与球体相交"""
    oc = ray_origin - sphere_center
    a = np.dot(ray_dir, ray_dir)
    b = 2.0 * np.dot(oc, ray_dir)
    c = np.dot(oc, oc) - sphere_radius * sphere_radius
    discriminant = b * b - 4 * a * c

    if discriminant < 0:
        return False

    # 检查交点是否在射线正方向
    t1 = (-b - math.sqrt(discriminant)) / (2 * a)
    t2 = (-b + math.sqrt(discriminant)) / (2 * a)

    return t1 > 1e-6 or t2 > 1e-6

def cylinder_silhouette_points(observer: np.ndarray, n_samples: int
= 72) -> List[np.ndarray]:
    """计算圆柱体轮廓关键点"""
    points = []

    # 底面圆周点
    for i in range(n_samples):
        angle = 2 * math.pi * i / n_samples
        x = TRUE_TARGET_CENTER[0] + CYLINDER_RADIUS * math.cos(angl

```

```

e)
    y = TRUE_TARGET_CENTER[1] + CYLINDER_RADIUS * math.sin(angl
e)
    points.append(np.array([x, y, TRUE_TARGET_CENTER[2]]))

# 顶面圆周点
for i in range(n_samples):
    angle = 2 * math.pi * i / n_samples
    x = TRUE_TARGET_CENTER[0] + CYLINDER_RADIUS * math.cos(angl
e)
    y = TRUE_TARGET_CENTER[1] + CYLINDER_RADIUS * math.sin(angl
e)
    points.append(np.array([x, y, TRUE_TARGET_CENTER[2] + CYLIND
ER_HEIGHT]))

# 侧面轮廓线切线点
obs_xy = np.array([observer[0], observer[1]])
center_xy = np.array([TRUE_TARGET_CENTER[0], TRUE_TARGET_CENTER
[1]])
to_obs = obs_xy - center_xy
dist_xy = np.linalg.norm(to_obs)

if dist_xy > CYLINDER_RADIUS:
    sin_alpha = CYLINDER_RADIUS / dist_xy
    cos_alpha = math.sqrt(1 - sin_alpha * sin_alpha)
    perp = np.array([-to_obs[1], to_obs[0]]) / dist_xy

    for sign in [-1, 1]:
        tangent_dir = cos_alpha * (to_obs / dist_xy) + sign * si
n_alpha * perp
        tangent_point_xy = center_xy + CYLINDER_RADIUS * tangent
_dir

        for z in np.linspace(TRUE_TARGET_CENTER[2], TRUE_TARGET_
CENTER[2] + CYLINDER_HEIGHT, 5):
            points.append(np.array([tangent_point_xy[0], tangent
_point_xy[1], z]))

    return points

class Problem4KnownParamsOptimizer:
    """基于已知最优参数的联合PSO优化器"""

    def __init__(self):

```

```

print("=== 问题 4: 基于已知最优参数的联合 PSO 优化 ===")
print("已知最优参数: ")
for uav, params in KNOWN_OPTIMAL.items():
    print(f" {uav}: {params['angle']:.2f}°, {params['speed']:.1f}m/s, {params['t_drop']:.2f}s, {params['tau']:.2f}s")
    print()

def uav_pos(self, uav_idx: int, theta: float, speed: float, t: float) -> np.ndarray:
    """无人机位置计算 (水平飞行) """
    direction = np.array([math.cos(theta), math.sin(theta), 0.0])

    pos = UAVS[uav_idx] + speed * t * direction
    pos[2] = UAVS[uav_idx][2] # 保持初始高度
    return pos

def projectile_at(self, uav_idx: int, theta: float, speed: float, t_drop: float, dt: float) -> Tuple[np.ndarray, np.ndarray]:
    """烟雾弹平抛运动"""
    P_rel = self.uav_pos(uav_idx, theta, speed, t_drop)
    direction = np.array([math.cos(theta), math.sin(theta), 0.0])

    v0 = speed * direction

    # 平抛运动公式
    pos = P_rel + v0 * dt + np.array([0.0, 0.0, -0.5 * G * dt * dt])

    vel = v0 + np.array([0.0, 0.0, -G * dt])
    return pos, vel

def detonation_state(self, uav_idx: int, theta: float, speed: float, t_drop: float, tau: float) -> Tuple[float, np.ndarray]:
    """计算起爆状态 (时间和位置) """
    t_det = t_drop + tau
    C0, _ = self.projectile_at(uav_idx, theta, speed, t_drop, tau)

    return t_det, C0

def cloud_center(self, uav_idx: int, theta: float, speed: float, t_drop: float, tau: float, t: float) -> np.ndarray:
    """云团中心位置 (起爆后匀速下沉) """
    t_det, C0 = self.detonation_state(uav_idx, theta, speed, t_drop, tau)

    if t < t_det:

```

```

        return C0
    dz = -CLOUD_DESCENT_V * (t - t_det)
    return C0 + np.array([0.0, 0.0, dz])

def m1_pos(self, t: float) -> np.ndarray:
    """导弹直线运动"""
    v_dir = unit(ORIGIN - M1_0)
    v = MISSILE_SPEED * v_dir
    return M1_0 + v * t

def is_completely_shielded_multi(self, observer: np.ndarray, cloud_positions: List[np.ndarray]) -> bool:
    """判断多个云团是否完全遮蔽圆柱体"""
    silhouette_points = cylinder_silhouette_points(observer, n_samples=72)

    for target_point in silhouette_points:
        ray_dir = unit(target_point - observer)
        ray_blocked = False

        # 检查是否被任何一个云团阻挡
        for cloud_pos in cloud_positions:
            if cloud_pos is not None and ray_sphere_intersection(
                observer, ray_dir, cloud_pos, CLOUD_RADIUS):
                ray_blocked = True
                break

        if not ray_blocked:
            return False # 存在未被遮挡的射线

    return True # 所有射线均被遮挡

def find_complete_shielding_intervals_multi(self, uav_params: List[dict], t0: float, t1: float, dt: float = 0.02) -> List[Tuple[float, float]]:
    """提取完全遮蔽的时间区间"""
    # 粗采样标记遮蔽状态
    times = np.arange(t0, t1 + 1e-9, dt)
    shielded = []

    for t in times:
        current_clouds = []
        for params in uav_params:
            t_det = params['t_drop'] + params['tau']

```

```

        t_end = t_det + CLOUD_EFFECT_TIME
        if t_det <= t <= t_end:
            cloud_pos = self.cloud_center(
                params['uav_idx'], params['theta'], params['
speed'],
                params['t_drop'], params['tau'], t
            )
            current_clouds.append(cloud_pos)
        else:
            current_clouds.append(None)

        observer = self.m1_pos(t)
        shielded.append(self.is_completely_shielded_multi(observer, current_clouds))

        # 提取连续遮蔽区间
        intervals = []
        in_interval = False
        start_time = t0

        for t, is_shielded in zip(times, shielded):
            if is_shielded and not in_interval:
                in_interval = True
                start_time = t
            elif not is_shielded and in_interval:
                in_interval = False
                intervals.append((start_time, t))

        if in_interval:
            intervals.append((start_time, t1))

        # 二分法精细化区间边界
        refined_intervals = []
        for start, end in intervals:
            # 优化起始边界
            left, right = max(t0, start - dt), start + dt
            for _ in range(20):
                mid = 0.5 * (left + right)
                current_clouds = self._get_current_clouds(uav_params, mid)

                if self.is_completely_shielded_multi(self.m1_pos(mid), current_clouds):
                    right = mid
            else:

```

```

        left = mid
        refined_start = right

        # 优化结束边界
        left, right = end - dt, min(t1, end + dt)
        for _ in range(20):
            mid = 0.5 * (left + right)
            current_clouds = self._get_current_clouds(uav_param
s, mid)
            if self.is_completely_shielded_multi(self.m1_pos(mi
d), current_clouds):
                left = mid
            else:
                right = mid
            refined_end = left

        if refined_end > refined_start:
            refined_intervals.append((refined_start, refined_en
d))

    return refined_intervals

def _get_current_clouds(self, uav_params: List[dict], t: float)
-> List[np.ndarray]:
    """ 获取指定时刻所有活跃云团的位置 """
    current_clouds = []
    for params in uav_params:
        t_det = params['t_drop'] + params['tau']
        t_end = t_det + CLOUD_EFFECT_TIME
        if t_det <= t <= t_end:
            cloud_pos = self.cloud_center(
                params['uav_idx'], params['theta'], params['spee
d'],
                params['t_drop'], params['tau'], t
            )
            current_clouds.append(cloud_pos)
        else:
            current_clouds.append(None)
    return current_clouds

def calculate_m1_total_shielding_time(self, params: np.ndarray)
-> float:
    """ 计算总遮蔽时间（使用参考代码的区间提取方法） """
    try:

```

```

        if len(params) != 12:
            return 0.0

        uav_params = []
        for i in range(3):
            base_idx = i * 4
            theta = params[base_idx]
            speed = params[base_idx + 1]
            t_drop = params[base_idx + 2]
            tau = params[base_idx + 3]

            # 约束检查
            if not (70 <= speed <= 140 and 0 <= t_drop <= 50 and
0 <= tau <= 15):
                return 0.0

            # 检查起爆点高度
            t_det, C0 = self.detonation_state(i, theta, speed, t
_drop, tau)

            if C0[2] <= 0:
                return 0.0

            uav_params.append({
                'uav_idx': i,
                'theta': theta,
                'speed': speed,
                't_drop': t_drop,
                'tau': tau
            })

            # 计算时间窗口
            t_det_list = [p['t_drop'] + p['tau'] for p in uav_param
s]

            t_end_list = [t + CLOUD_EFFECT_TIME for t in t_det_list]

            t_start = min(t_det_list)
            t_end = max(t_end_list)

            # 提取遮蔽区间并计算总时长
            intervals = self.find_complete_shielding_intervals_multi
(uav_params, t_start, t_end)
            total_time = sum(b - a for a, b in intervals)

        return total_time

```



```

except Exception as e:
    print(f"计算错误: {str(e)}")
    return 0.0

def known_params_pso_optimize(self, n_particles=100, n_iteration
s=200):
    """基于已知最优参数的PSO 优化"""
    print("=== 基于已知最优参数的 PSO 优化 ===")
    print(f"PSO 参数: {n_particles}个粒子, {n_iterations}次迭代")
    print()

    # 转换已知参数为弧度
    fy1_known = [math.radians(KNOWN_OPTIMAL['FY1']['angle']),
                  KNOWN_OPTIMAL['FY1']['speed'],
                  KNOWN_OPTIMAL['FY1']['t_drop'],
                  KNOWN_OPTIMAL['FY1']['tau']]

    fy2_known = [math.radians(KNOWN_OPTIMAL['FY2']['angle']),
                  KNOWN_OPTIMAL['FY2']['speed'],
                  KNOWN_OPTIMAL['FY2']['t_drop'],
                  KNOWN_OPTIMAL['FY2']['tau']]

    fy3_known = [math.radians(KNOWN_OPTIMAL['FY3']['angle']),
                  KNOWN_OPTIMAL['FY3']['speed'],
                  KNOWN_OPTIMAL['FY3']['t_drop'],
                  KNOWN_OPTIMAL['FY3']['tau']]

    # 设置搜索范围
    angle_range = math.radians(10) # ±10 度
    speed_range = 15 # ±15 m/s
    t_drop_range = 3 # ±3 秒
    tau_range = 2 # ±2 秒

    bounds_min = np.array([
        fy1_known[0] - angle_range, fy1_known[1] - speed_range,
        max(0, fy1_known[2] - t_drop_range), max(0, fy1_known[3]
- tau_range),
        fy2_known[0] - angle_range, fy2_known[1] - speed_range,
        max(0, fy2_known[2] - t_drop_range), max(0, fy2_known[3]
- tau_range),

```

```

        fy3_known[0] - angle_range, fy3_known[1] - speed_range,
        max(0, fy3_known[2] - t_drop_range), max(0, fy3_known[3]
- tau_range)
    ])

    bounds_max = np.array([
        fy1_known[0] + angle_range, min(140, fy1_known[1] + spee
d_range),
        fy1_known[2] + t_drop_range, min(15, fy1_known[3] + tau_
range),

        fy2_known[0] + angle_range, min(140, fy2_known[1] + spee
d_range),
        fy2_known[2] + t_drop_range, min(15, fy2_known[3] + tau_
range),

        fy3_known[0] + angle_range, min(140, fy3_known[1] + spee
d_range),
        fy3_known[2] + t_drop_range, min(15, fy3_known[3] + tau_
range)
    ])

    print("搜索范围: ")
    print(f"  角度:  $\pm\{\text{math.degrees(angle\_range):.0f}\}^\circ$  | 速度:  $\pm\{\text{s
peed\_range}\}\text{m/s}$ ")
    print(f"  投放时间:  $\pm\{\text{t\_drop\_range}\}\text{s}$  | 延时:  $\pm\{\text{tau\_range}\}\text{s}$ ")
    print()

    # 初始化粒子群
    particles = np.zeros((n_particles, 12))
    known_optimal_params = np.concatenate([fy1_known, fy2_known,
fy3_known])

    for i in range(n_particles):
        if i == 0:
            # 初始粒子使用已知最优参数
            particles[i] = known_optimal_params
        elif i < n_particles // 2:
            # 50%粒子在已知最优附近小幅扰动
            particles[i] = known_optimal_params + np.random.unif
orm(-0.1, 0.1, 12) * (bounds_max - bounds_min)
        else:
            # 50%粒子在搜索范围内随机分布

```

```

        particles[i] = np.random.uniform(bounds_min, bounds_
max)

particles = np.clip(particles, bounds_min, bounds_max)
velocities = np.random.uniform(-0.01, 0.01, (n_particles, 1
2))

# PSO 状态变量
personal_best = particles.copy()
personal_best_scores = np.full(n_particles, -np.inf)
global_best = None
global_best_score = -np.inf

print("开始 PSO 优化...")
start_time = time.time()

for iteration in range(n_iterations):
    # 动态调整 PSO 参数
    w = 0.8 - 0.5 * iteration / n_iterations
    c1 = 2.0 - 1.0 * iteration / n_iterations
    c2 = 1.0 + 1.0 * iteration / n_iterations

    # 评估所有粒子
    for i in range(n_particles):
        score = self.calculate_m1_total_shielding_time(parti
cles[i])

        # 更新个体最优
        if score > personal_best_scores[i]:
            personal_best_scores[i] = score
            personal_best[i] = particles[i].copy()

        # 更新全局最优
        if score > global_best_score:
            global_best_score = score
            global_best = particles[i].copy()

        # 显示优化进度
        fy1_params = particles[i][:4]
        fy2_params = particles[i][4:8]
        fy3_params = particles[i][8:12]

        print(f" 迭代{iteration+1}: 新最优 → {score:.5f}
s")

```

```

        print(f"    FY1: {math.degrees(fy1_params[0]):.2f}°, {fy1_params[1]:.1f}m/s, {fy1_params[2]:.2f}s, {fy1_params[3]:.2f}s")

        print(f"    FY2: {math.degrees(fy2_params[0]):.2f}°, {fy2_params[1]:.1f}m/s, {fy2_params[2]:.2f}s, {fy2_params[3]:.2f}s")

        print(f"    FY3: {math.degrees(fy3_params[0]):.2f}°, {fy3_params[1]:.1f}m/s, {fy3_params[2]:.2f}s, {fy3_params[3]:.2f}s")

# 更新粒子位置和速度
for i in range(n_particles):
    r1, r2 = np.random.random(12), np.random.random(12)
    velocities[i] = (w * velocities[i] +
                    c1 * r1 * (personal_best[i] - particles[i]) +
                    c2 * r2 * (global_best - particles[i]))
    particles[i] += velocities[i]
    particles[i] = np.clip(particles[i], bounds_min, bounds_max)

# 定期输出进度
if (iteration + 1) % 25 == 0:
    elapsed = time.time() - start_time
    print(f"    迭代{iteration+1}/{n_iterations}: 当前最优={global_best_score:.5f}s, 耗时={elapsed:.1f}s")

total_time = time.time() - start_time
print(f"PSO 优化完成 (总耗时: {total_time:.2f}s)")

# 整理优化结果
result = {
    'total_score': global_best_score,
    'uavs': []
}

uav_names = ['FY1', 'FY2', 'FY3']
for i in range(3):
    base_idx = i * 4
    uav_result = {
        'uav_name': uav_names[i],
        'theta': global_best[base_idx],
        'speed': global_best[base_idx + 1],
    }

```

```

        't_drop': global_best[base_idx + 2],
        'tau': global_best[base_idx + 3],
        'angle_deg': math.degrees(global_best[base_idx])
    }
    result['uavs'].append(uav_result)

    return result

def main():
    print("程序开始运行...")
    try:
        optimizer = Problem4KnownParamsOptimizer()
        result = optimizer.known_params_pso_optimize(n_particles=50,
n_iterations=100)
    except Exception as e:
        print(f"程序出错: {e}")
        import traceback
        traceback.print_exc()
        return

    # 输出最终结果
    print("\n" + "="*70)
    print("问题 4 多机单弹: 基于已知最优参数的联合优化结果")
    print("="*70)

    if result['total_score'] > 0:
        print(f"M1 总遮蔽时间: {result['total_score']:.5f} 秒")
        print()

        for uav in result['uavs']:
            print(f"{uav['uav_name']}最优策略: ")
            print(f"  飞行方向: {uav['angle_deg']:.2f}°")
            print(f"  飞行速度: {uav['speed']:.2f} m/s")
            print(f"  投放时间: {uav['t_drop']:.3f} s")
            print(f"  起爆延时: {uav['tau']:.3f} s")
            print(f"  起爆时间: {uav['t_drop'] + uav['tau']:.3f} s")
            print()

    # 保存结果
    with open('problem4_result.txt', 'w', encoding='utf-8') as
f:

        f.write("问题 4: 基于已知最优参数的联合优化结果\n")
        f.write("="*50 + "\n")

```

```

f.write(f"M1 总遮蔽时间: {result['total_score']:.5f} 秒\n\n")

for uav in result['uavs']:
    f.write(f"{uav['uav_name']}最优策略: \n")
    f.write(f" 飞行方向: {uav['angle_deg']:.2f}°\n")
    f.write(f" 飞行速度: {uav['speed']:.2f} m/s\n")
    f.write(f" 投放时间: {uav['t_drop']:.3f} s\n")
    f.write(f" 起爆延时: {uav['tau']:.3f} s\n")
    f.write(f" 起爆时间: {uav['t_drop'] + uav['tau']:.3f} s\n\n")

f.write(f"协同效率: {efficiency:.1f}%\n")

print("结果已保存到: problem4_result.txt")

else:
    print("未找到有效的协同策略")

print("="*70)

if __name__ == "__main__":
    random.seed(42)
    np.random.seed(42)
    main()

```

附录 E 问题五源代码

```

import math
import numpy as np
from typing import List, Tuple, Dict
import time
import pandas as pd

# 物理常量
G = 9.8
MISSILE_SPEED = 300.0
CLOUD_DESCENT_V = 3.0
CLOUD_RADIUS = 10.0
CLOUD_EFFECT_TIME = 20.0

# 关键点坐标
ORIGIN = np.array([0.0, 0.0, 0.0])

```

```

TRUE_TARGET_CENTER = np.array([0.0, 200.0, 0.0])
CYLINDER_RADIUS = 7.0
CYLINDER_HEIGHT = 10.0

# 初始位置
UAV_POSITIONS = {
    'FY1': np.array([17800.0, 0.0, 1800.0]),
    'FY2': np.array([12000.0, 1400.0, 1400.0]),
    'FY3': np.array([6000.0, -3000.0, 700.0]),
    'FY4': np.array([11000.0, 2000.0, 1800.0]),
    'FY5': np.array([13000.0, -2000.0, 1300.0])
}

MISSILE_POSITIONS = {
    'M1': np.array([20000.0, 0.0, 2000.0]),
    'M2': np.array([19000.0, 600.0, 2100.0]),
    'M3': np.array([18000.0, -600.0, 1900.0])
}

def unit(v: np.ndarray) -> np.ndarray:
    n = np.linalg.norm(v)
    return v / n if n > 0 else v

class Problem5MultiUAVOptimizer:
    def __init__(self):
        print("=== 问题 5: 多无人机协同烟幕干扰优化 ===")
        print("5 架无人机对 3 枚导弹的协同干扰策略")
        print("FY1→M1, FY2&FY4→M2, FY3&FY5→M3")
        print("基于已知的三弹最优解进行全局 PSO 优化")
        print()

        self.known_solutions = {
            'FY1_M1': {
                'angle': 7.79, 'speed': 111.0,
                'drops': [0.00, 1.00, 2.00], 'delays': [0.00, 0.00,
0.27],
                'score': 6.40313
            },
            'FY2_M2': {
                'angle': 297.21, 'speed': 134.9,
                'drops': [6.59, 8.90, 10.22], 'delays': [1.44, 1.59,
1.64],
                'score': 3.94141
            },
        },

```

```

        'FY4_M2': {
            'angle': 283.53, 'speed': 121.6,
            'drops': [3.07, 5.08, 7.66], 'delays': [10.45, 6.97,
6.81],
            'score': 3.75195
        },
        'FY3_M3': {
            'angle': 79.93, 'speed': 138.9,
            'drops': [12.49, 16.98, 19.82], 'delays': [1.94, 1.2
1, 1.14],
            'score': 3.04189
        },
        'FY5_M3': {
            'angle': 122.11, 'speed': 131.8,
            'drops': [12.05, 14.68, 16.06], 'delays': [2.44, 2.5
5, 3.42],
            'score': 3.67734
        }
    }
}

```

```

def uav_pos(self, uav_name: str, theta: float, speed: float, t:
float) -> np.ndarray:
    direction = np.array([math.cos(theta), math.sin(theta), 0.
0])

    pos = UAV_POSITIONS[uav_name] + speed * t * direction
    pos[2] = UAV_POSITIONS[uav_name][2] # 保持初始高度
    return pos

def missile_pos(self, missile_name: str, t: float) -> np.ndarra
y:
    v_dir = unit(ORIGIN - MISSILE_POSITIONS[missile_name])
    v = MISSILE_SPEED * v_dir
    return MISSILE_POSITIONS[missile_name] + v * t

def cloud_center(self, uav_name: str, theta: float, speed: floa
t, t_drop: float, tau: float, t: float) -> np.ndarray:
    t_det = t_drop + tau
    if t < t_det:
        return np.array([0, 0, -1000])
    drop_pos = self.uav_pos(uav_name, theta, speed, t_drop)
    direction = np.array([math.cos(theta), math.sin(theta), 0.
0])

    v0 = speed * direction
    explode_pos = drop_pos + v0 * tau + np.array([0.0, 0.0, -0.5

```



```

* G * tau * tau])
    dt_after = t - t_det
    if dt_after > CLOUD_EFFECT_TIME:
        return np.array([0, 0, -1000])
    return explode_pos + np.array([0.0, 0.0, -CLOUD_DESCENT_V *
dt_after])

def ray_sphere_intersection(self, ray_origin: np.ndarray, ray_dir:
np.ndarray, sphere_center: np.ndarray, sphere_radius: float) -> bool:
    if sphere_center[2] < -500:
        return False
    oc = ray_origin - sphere_center
    a = np.dot(ray_dir, ray_dir)
    b = 2.0 * np.dot(oc, ray_dir)
    c = np.dot(oc, oc) - sphere_radius * sphere_radius
    discriminant = b * b - 4 * a * c
    if discriminant < 0:
        return False
    t1 = (-b - math.sqrt(discriminant)) / (2 * a)
    t2 = (-b + math.sqrt(discriminant)) / (2 * a)
    return t1 > 1e-6 or t2 > 1e-6

def cylinder_silhouette_points(self, observer: np.ndarray, n_samples: int = 36) -> List[np.ndarray]:
    points = []
    for i in range(n_samples):
        angle = 2 * math.pi * i / n_samples
        x = TRUE_TARGET_CENTER[0] + CYLINDER_RADIUS * math.cos(angle)
        y = TRUE_TARGET_CENTER[1] + CYLINDER_RADIUS * math.sin(angle)
        points.append(np.array([x, y, TRUE_TARGET_CENTER[2]]))
        points.append(np.array([x, y, TRUE_TARGET_CENTER[2] + CYLINDER_HEIGHT]))
    obs_xy = observer[:2]
    center_xy = np.array([TRUE_TARGET_CENTER[0], TRUE_TARGET_CENTER[1]])
    to_obs = obs_xy - center_xy
    dist_xy = np.linalg.norm(to_obs)
    if dist_xy > CYLINDER_RADIUS:
        sin_alpha = CYLINDER_RADIUS / dist_xy
        cos_alpha = math.sqrt(1 - sin_alpha * sin_alpha)
        perp = np.array([-to_obs[1], to_obs[0]]) / dist_xy

```

```

        for sign in [-1, 1]:
            tangent_dir = cos_alpha * (to_obs / dist_xy) + sign
            * sin_alpha * perp
            tangent_point_xy = center_xy + CYLINDER_RADIUS * tan
            gent_dir
            for z in np.linspace(TRUE_TARGET_CENTER[2], TRUE_TAR
            GET_CENTER[2] + CYLINDER_HEIGHT, 5):
                points.append(np.array([tangent_point_xy[0], tan
            gent_point_xy[1], z]))
        return points

```

```

    def is_completely_shielded_multi(self, observer: np.ndarray, clo
    ud_positions: List[np.ndarray]) -> bool:
        silhouette_points = self.cylinder_silhouette_points(observer
        r, n_samples=36)
        for target_point in silhouette_points:
            ray_dir = unit(target_point - observer)
            ray_blocked = False
            for cloud_pos in cloud_positions:
                if cloud_pos is not None and self.ray_sphere_interse
            ction(observer, ray_dir, cloud_pos, CLOUD_RADIUS):
                    ray_blocked = True
                    break
            if not ray_blocked:
                return False
        return True

```

```

    def calculate_single_missile_shielding(self, missile_name: str,
    uav_configs: List[Tuple[str, np.ndarray]], dt: float = 0.05) -> floa
    t:

```

```

        all_clouds = []
        for uav_name, params in uav_configs:
            theta, speed, t1, tau1, t2, tau2, t3, tau3 = params
            for t_drop, tau in [(t1, tau1), (t2, tau2), (t3, tau3)]:

                t_det, C0 = self.detonation_state(uav_name, theta, s
            peed, t_drop, tau)
                if C0[2] <= 50: # 起爆高度约束
                    return 0.0
                all_clouds.append({
                    'uav': uav_name,
                    'theta': theta,
                    'speed': speed,
                    't_drop': t_drop,

```

```

        'tau': tau,
        't_start': t_det,
        't_end': t_det + CLOUD_EFFECT_TIME
    })

    if not all_clouds:
        return 0.0

    t_start = min(cloud['t_start'] for cloud in all_clouds)
    t_end = max(cloud['t_end'] for cloud in all_clouds)
    times = np.arange(t_start, t_end + 1e-9, dt)
    shielded = []

    for t in times:
        current_clouds = []
        for cloud in all_clouds:
            if cloud['t_start'] <= t <= cloud['t_end']:
                cloud_pos = self.cloud_center(cloud['uav'], cloud
d['theta'], cloud['speed'], cloud['t_drop'], cloud['tau'], t)
                current_clouds.append(cloud_pos)
            else:
                current_clouds.append(None)
        observer = self.missile_pos(missile_name, t)
        shielded.append(self.is_completely_shielded_multi(observ
er, current_clouds))

    intervals = []
    in_interval = False
    start_time = t_start
    for t, is_shielded in zip(times, shielded):
        if is_shielded and not in_interval:
            in_interval = True
            start_time = t
        elif not is_shielded and in_interval:
            in_interval = False
            intervals.append((start_time, t))
    if in_interval:
        intervals.append((start_time, t_end))

    refined_intervals = []
    for start, end in intervals:
        left, right = max(t_start, start - dt), start + dt
        for _ in range(20):
            mid = 0.5 * (left + right)

```

```

        current_clouds = [self.cloud_center(cloud['uav'], cloud['theta'], cloud['speed'], cloud['t_drop'], cloud['tau'], mid)
                           if cloud['t_start'] <= mid <= cloud['t_end'] else None for cloud in all_clouds]
        if self.is_completely_shielded_multi(self.missile_pos(missile_name, mid), current_clouds):
            right = mid
        else:
            left = mid
        refined_start = right

    left, right = end - dt, min(t_end, end + dt)
    for _ in range(20):
        mid = 0.5 * (left + right)
        current_clouds = [self.cloud_center(cloud['uav'], cloud['theta'], cloud['speed'], cloud['t_drop'], cloud['tau'], mid)
                           if cloud['t_start'] <= mid <= cloud['t_end'] else None for cloud in all_clouds]
        if self.is_completely_shielded_multi(self.missile_pos(missile_name, mid), current_clouds):
            left = mid
        else:
            right = mid
        refined_end = left

    if refined_end > refined_start:
        refined_intervals.append((refined_start, refined_end))

    return sum(b - a for a, b in refined_intervals)

def detonation_state(self, uav_name: str, theta: float, speed: float, t_drop: float, tau: float) -> Tuple[float, np.ndarray]:
    t_det = t_drop + tau
    drop_pos = self.uav_pos(uav_name, theta, speed, t_drop)
    direction = np.array([math.cos(theta), math.sin(theta), 0.0])
    v0 = speed * direction
    explode_pos = drop_pos + v0 * tau + np.array([0.0, 0.0, -0.5 * G * tau * tau])
    return t_det, explode_pos

def calculate_total_shielding_time(self, params: np.ndarray) -> float:

```

```

try:
    fy1_params = params[0:8]
    fy2_params = params[8:16]
    fy4_params = params[16:24]
    fy3_params = params[24:32]
    fy5_params = params[32:40]

    all_params = [fy1_params, fy2_params, fy4_params, fy3_pa
rams, fy5_params]
    for i, uav_params in enumerate(all_params):
        theta, speed, t1, tau1, t2, tau2, t3, tau3 = uav_par
ams

        if not (70 <= speed <= 140):
            return 0.0
        drops = [t1, t2, t3]
        if not all(drops[j] <= drops[j+1] for j in range(len
(drops)-1)):
            return 0.0
        for j in range(len(drops)-1):
            if drops[j+1] - drops[j] < 1.0:
                return 0.0
        for tau in [tau1, tau2, tau3]:
            if not (0 <= tau <= 15):
                return 0.0
        for t_drop in drops:
            if not (0 <= t_drop <= 30):
                return 0.0

    total_score = 0.0
    m1_time = self.calculate_single_missile_shielding('M1',
[('FY1', fy1_params)])
    m2_time = self.calculate_single_missile_shielding('M2',
[('FY2', fy2_params), ('FY4', fy4_params)])
    m3_time = self.calculate_single_missile_shielding('M3',
[('FY3', fy3_params), ('FY5', fy5_params)])

    # 协同奖励: 鼓励 M2 和 M3 的遮蔽时间接近各自已知解之和
    m2_known = self.known_solutions['FY2_M2']['score'] + sel
f.known_solutions['FY4_M2']['score']
    m3_known = self.known_solutions['FY3_M3']['score'] + sel
f.known_solutions['FY5_M3']['score']
    m2_bonus = min(m2_time / m2_known, 1.0) * 0.1 # 最多加
0.1 秒

    m3_bonus = min(m3_time / m3_known, 1.0) * 0.1

```

```

        total_score = m1_time + m2_time + m3_time + m2_bonus + m
3_bonus

        return total_score
    except:
        return 0.0

    def local_refinement(self, best_params: np.ndarray) -> Tuple[float, np.ndarray]:
        print("执行局部精细优化...")
        best_fitness = self.calculate_total_shielding_time(best_params)
        best_params = best_params.copy()

        uav_names = ['FY1', 'FY2', 'FY4', 'FY3', 'FY5']
        for i, uav_name in enumerate(uav_names):
            base_idx = i * 8
            theta0, speed0, t1_0, tau1_0, t2_0, tau2_0, t3_0, tau3_0
            = best_params[base_idx:base_idx+8]

            theta_samples = np.linspace(max(0, theta0 - 0.1), min(2*
math.pi, theta0 + 0.1), 3)
            speed_samples = np.linspace(max(70, speed0 - 5), min(14
0, speed0 + 5), 3)
            t1_samples = np.linspace(max(0, t1_0 - 1), min(30, t1_0
+ 1), 3)
            tau1_samples = np.linspace(max(0, tau1_0 - 0.5), min(15,
tau1_0 + 0.5), 3)
            t2_samples = np.linspace(max(t1_0 + 1, t2_0 - 1), min(3
0, t2_0 + 1), 3)
            tau2_samples = np.linspace(max(0, tau2_0 - 0.5), min(15,
tau2_0 + 0.5), 3)
            t3_samples = np.linspace(max(t2_0 + 1, t3_0 - 1), min(3
0, t3_0 + 1), 3)
            tau3_samples = np.linspace(max(0, tau3_0 - 0.5), min(15,
tau3_0 + 0.5), 3)

            for theta in theta_samples:
                for speed in speed_samples:
                    for t1 in t1_samples:
                        for tau1 in tau1_samples:
                            for t2 in t2_samples:
                                for tau2 in tau2_samples:
                                    for t3 in t3_samples:

```

```

        for tau3 in tau3_samples:
            test_params = best_param

s.copy()

            test_params[base_idx:base_idx+8] = [theta, speed, t1, tau1, t2, tau2, t3, tau3]
            fitness = self.calculate
            _total_shielding_time(test_params)

            if fitness > best_fitness:

s:
                best_fitness = fitness

ss

            best_params[base_idx:base_idx+8] = [theta, speed, t1, tau1, t2, tau2, t3, tau3]
            print(f" 新最优 ({uav_name}): {fitness:.5f}s")

```

```

    return best_fitness, best_params

```

```

def pso_optimize_problem5(self, n_particles=200, n_iterations=200):

```

```

    print("=== 问题 5 PSO 优化开始 ===")
    print(f"粒子数: {n_particles}, 迭代次数: {n_iterations}")
    print("优化 40 个参数: 5 架无人机 × 8 参数/架")
    print()

```

```

    base_solutions = [
        self.known_solutions['FY1_M1'],
        self.known_solutions['FY2_M2'],
        self.known_solutions['FY4_M2'],
        self.known_solutions['FY3_M3'],
        self.known_solutions['FY5_M3']
    ]

```

```

    bounds_min = []
    bounds_max = []
    for sol in base_solutions:
        angle_rad = math.radians(sol['angle'])
        bounds_min.extend([
            max(0, angle_rad - 0.1745), # ±10°
            max(70, sol['speed'] - 10),
            max(0, sol['drops'][0] - 2),
            max(0, sol['delays'][0] - 1),
            max(0, sol['drops'][1] - 2),

```

```

        max(0, sol['delays'][1] - 1),
        max(0, sol['drops'][2] - 2),
        max(0, sol['delays'][2] - 1)
    ])
    bounds_max.extend([
        min(2*math.pi, angle_rad + 0.1745),
        min(140, sol['speed'] + 10),
        min(30, sol['drops'][0] + 2),
        min(15, sol['delays'][0] + 1),
        min(30, sol['drops'][1] + 2),
        min(15, sol['delays'][1] + 1),
        min(30, sol['drops'][2] + 2),
        min(15, sol['delays'][2] + 1)
    ])

    bounds_min = np.array(bounds_min)
    bounds_max = np.array(bounds_max)

    particles = np.zeros((n_particles, 40))
    for i in range(int(n_particles * 0.9)):
        for j, sol in enumerate(base_solutions):
            base_idx = j * 8
            particles[i, base_idx] = math.radians(sol['angle'])
            + np.random.uniform(-0.3, 0.3)
            particles[i, base_idx + 1] = sol['speed'] + np.random.
            m.uniform(-7, 7)
            for k in range(3):
                particles[i, base_idx + 2 + k*2] = sol['drops']
                [k] + np.random.uniform(-1.5, 1.5)
                particles[i, base_idx + 3 + k*2] = sol['delays']
                [k] + np.random.uniform(-0.7, 0.7)

    for i in range(int(n_particles * 0.9), n_particles):
        particles[i] = np.random.uniform(bounds_min, bounds_max)

        for j in range(5):
            base_idx = j * 8
            drops = [particles[i, base_idx + 2], particles[i, ba
            se_idx + 4], particles[i, base_idx + 6]]
            drops_sorted = sorted(drops)
            particles[i, base_idx + 2], particles[i, base_idx +
            4], particles[i, base_idx + 6] = drops_sorted

    particles = np.clip(particles, bounds_min, bounds_max)

```



```

        velocities = np.random.uniform(-0.05, 0.05, (n_particles, 4
0))

    personal_best = particles.copy()
    personal_best_scores = np.full(n_particles, -np.inf)
    global_best = particles[0].copy()
    global_best_score = -np.inf

    print("开始 PSO 优化...")
    start_time = time.time()

    for iteration in range(n_iterations):
        w = 0.9 - 0.7 * iteration / n_iterations
        c1 = 2.5 - 1.8 * iteration / n_iterations
        c2 = 0.5 + 1.8 * iteration / n_iterations

        for i in range(n_particles):
            score = self.calculate_total_shielding_time(particle
s[i])

            if score > personal_best_scores[i]:
                personal_best_scores[i] = score
                personal_best[i] = particles[i].copy()
            if score > global_best_score:
                global_best_score = score
                global_best = particles[i].copy()
                print(f"  迭代{iteration+1}: 新最优 → {score:.5f}
s")

                for j, uav_name in enumerate(['FY1', 'FY2', 'FY4
', 'FY3', 'FY5']):
                    base_idx = j * 8
                    theta, speed, t1, tau1, t2, tau2, t3, tau3 =
particles[i][base_idx:base_idx+8]
                    print(f"      {uav_name}: {math.degrees(thet
a):.2f}°, {speed:.1f}m/s, "
                        f"投放[{t1:.2f},{t2:.2f},{t3:.2f}], 延
时[{tau1:.2f},{tau2:.2f},{tau3:.2f}]")

                for i in range(n_particles):
                    r1, r2 = np.random.random(40), np.random.random(40)
                    velocities[i] = (w * velocities[i] +
                                c1 * r1 * (personal_best[i] - particl
es[i]) +
                                c2 * r2 * (global_best - particles
[i]))

                    particles[i] += velocities[i]

```

```

        particles[i] = np.clip(particles[i], bounds_min, bounds_max)

        for j in range(5):
            base_idx = j * 8
            drops = [particles[i, base_idx + 2], particles[i, base_idx + 4], particles[i, base_idx + 6]]
            drops_sorted = sorted(drops)
            particles[i, base_idx + 2], particles[i, base_idx + 4], particles[i, base_idx + 6] = drops_sorted

            if (iteration + 1) % 20 == 0:
                elapsed = time.time() - start_time
                print(f" 迭代{iteration+1}/{n_iterations}: 最优={global_best_score:.5f}s, 耗时={elapsed:.1f}s")

        final_score, final_params = self.local_refinement(global_best_score, final_params)

        total_time = time.time() - start_time
        print(f"PSO 优化完成 (总耗时: {total_time:.2f}s)")

        return final_params, final_score

def save_results_to_excel(self, optimal_params: np.ndarray, filename: str = "result5.xlsx"):
    if optimal_params is None:
        print("无有效结果可保存")
        return None

    uav_names = ['FY1', 'FY2', 'FY4', 'FY3', 'FY5']
    target_missiles = ['M1', 'M2', 'M2', 'M3', 'M3']
    results = []

    for i, (uav_name, target) in enumerate(zip(uav_names, target_missiles)):
        base_idx = i * 8
        theta, speed, t1, tau1, t2, tau2, t3, tau3 = optimal_params[base_idx:base_idx+8]

        for j, (t_drop, tau) in enumerate([(t1, tau1), (t2, tau2), (t3, tau3)]):
            drop_pos = self.uav_pos(uav_name, theta, speed, t_drop)

            direction = np.array([math.cos(theta), math.sin(theta), 0.0])

            v0 = speed * direction
            explode_pos = drop_pos + v0 * tau + np.array([0.0, 0.0, -0.5 * G * tau * tau])

```

```

        results.append({
            '无人机编号': uav_name,
            '目标导弹': target,
            '烟幕弹编号': j + 1,
            '飞行方向(度)': math.degrees(theta),
            '飞行速度(m/s)': speed,
            '投放时间(s)': t_drop,
            '起爆延时(s)': tau,
            '起爆时间(s)': t_drop + tau,
            '投放点 X(m)': drop_pos[0],
            '投放点 Y(m)': drop_pos[1],
            '投放点 Z(m)': drop_pos[2],
            '起爆点 X(m)': explode_pos[0],
            '起爆点 Y(m)': explode_pos[1],
            '起爆点 Z(m)': explode_pos[2]
        })
    df = pd.DataFrame(results)
    df.to_excel(filename, index=False)
    print(f"结果已保存到 {filename}")
    return df

def analyze_results(self, optimal_params: np.ndarray):
    if optimal_params is None:
        return
    print("\n" + "="*80)
    print("问题 5 最优解分析")
    print("="*80)
    uav_names = ['FY1', 'FY2', 'FY4', 'FY3', 'FY5']
    target_missiles = ['M1', 'M2', 'M2', 'M3', 'M3']
    total_score = self.calculate_total_shielding_time(optimal_
rams)
    print(f"总遮蔽时间: {total_score:.5f} 秒")
    print()
    for i, (uav_name, target) in enumerate(zip(uav_names, target
_missiles)):
        base_idx = i * 8
        theta, speed, t1, tau1, t2, tau2, t3, tau3 = optimal_par
ams[base_idx:base_idx+8]
        print(f" [{uav_name} → {target}] ")
        print(f" 飞行方向: {math.degrees(theta):.2f}°")
        print(f" 飞行速度: {speed:.1f} m/s")
        print(f" 投放时间: [{t1:.2f}, {t2:.2f}, {t3:.2f}] s")
        print(f" 起爆延时: [{tau1:.2f}, {tau2:.2f}, {tau3:.2f}]
s")

```

```

        print(f" 起爆时间: [{t1+tau1:.2f}, {t2+tau2:.2f}, {t3+ta
u3:.2f}] s")
        intervals = [t2-t1, t3-t2]
        min_interval = min(intervals)
        speed_ok = 70 <= speed <= 140
        interval_ok = min_interval >= 1.0

        print(f" 约束检查: 速度{'✓' if speed_ok else 'X'}, 间隔
{'✓' if interval_ok else 'X'}({min_interval:.2f}s)")
        print()
        print("各导弹遮蔽时间:")
        fy1_params = optimal_params[0:8]
        m1_time = self.calculate_single_missile_shielding('M1', [('F
Y1', fy1_params)])
        print(f" M1 (FY1): {m1_time:.5f} s")
        fy2_params = optimal_params[8:16]
        fy4_params = optimal_params[16:24]
        m2_time = self.calculate_single_missile_shielding('M2', [('F
Y2', fy2_params), ('FY4', fy4_params)])
        print(f" M2 (FY2+FY4): {m2_time:.5f} s")
        fy3_params = optimal_params[24:32]
        fy5_params = optimal_params[32:40]
        m3_time = self.calculate_single_missile_shielding('M3', [('F
Y3', fy3_params), ('FY5', fy5_params)])
        print(f" M3 (FY3+FY5): {m3_time:.5f} s")
        print(f" 总计: {m1_time + m2_time + m3_time:.5f} s")

def main():
    optimizer = Problem5MultiUAVOptimizer()
    print("已知最优三弹解:")
    for key, sol in optimizer.known_solutions.items():
        print(f" {key}: {sol['angle']:.2f}°, {sol['speed']:.1f}m/s
→ {sol['score']:.5f}s")
    print()
    start_time = time.time()
    optimal_params, optimal_score = optimizer.pso_optimize_problem5
(n_particles=200, n_iterations=200)
    total_time = time.time() - start_time
    if optimal_params is not None:
        optimizer.analyze_results(optimal_params)
        df = optimizer.save_results_to_excel(optimal_params, "result
5.xlsx")
    print(f"\n 优化总用时: {total_time:.2f} 秒")
    print("="*80)

```

```

        print("\nExcel 文件预览 (前 10 行):")
        print(df.head(10).to_string(index=False))
    else:
        print("优化失败, 未找到有效解")

if __name__ == "__main__":
    np.random.seed(42)
    main()

```

附录 F 参数测试代码

```

"""
测试指定参数的遮蔽时间
"""

import math
import numpy as np
from typing import List, Tuple

# 物理常量
G = 9.8
MISSILE_SPEED = 300.0
CLOUD_DESCENT_V = 3.0
CLOUD_RADIUS = 10.0
CLOUD_EFFECT_TIME = 20.0

# 关键点坐标
ORIGIN = np.array([0.0, 0.0, 0.0]) # 假目标
TRUE_TARGET_CENTER = np.array([0.0, 200.0, 0.0]) # 真目标圆柱底面圆心
CYLINDER_RADIUS = 7.0
CYLINDER_HEIGHT = 10.0

# 初始状态
M1_0 = np.array([19000, 600, 2100])
FY1_0 = np.array([12000, 1400, 1400])

# 测试参数: 自主输入
TEST_THETA = math.radians(296.52)
TEST_SPEED = 139.3
TEST_T_DROP = 8.88 # 投弹时间
TEST_TAU = 2.41 # 起爆延时

def unit(v: np.ndarray) -> np.ndarray:
    n = np.linalg.norm(v)

```

```

    return v / n if n > 0 else v

def uav_pos_test(t: float) -> np.ndarray:
    """使用测试参数的无人机位置"""
    direction = np.array([math.cos(TEST_THETA), math.sin(TEST_THETA),
    A), 0.0])
    return FY1_0 + TEST_SPEED * t * direction

def projectile_at_test(dt: float) -> Tuple[np.ndarray, np.ndarray]:
    """使用测试参数的烟幕弹平抛运动"""
    P_rel = uav_pos_test(TEST_T_DROP)
    # 水平速度与FY一致
    direction = np.array([math.cos(TEST_THETA), math.sin(TEST_THETA),
    A), 0.0])
    v0 = TEST_SPEED * direction

    # 平抛运动
    pos = P_rel + v0 * dt + np.array([0.0, 0.0, -0.5 * G * dt * dt])

    vel = v0 + np.array([0.0, 0.0, -G * dt])
    return pos, vel

def detonation_state_test() -> Tuple[float, np.ndarray]:
    """返回起爆的绝对时间 t_det 以及起爆点坐标 C0"""
    t_det = TEST_T_DROP + TEST_TAU
    C0, _ = projectile_at_test(TEST_TAU)
    return t_det, C0

def cloud_center_test(t: float) -> np.ndarray:
    """云团中心位置: 起爆后以3 m/s 匀速下沉"""
    t_det, C0 = detonation_state_test()
    if t < t_det:
        return C0 # 未起爆
    dz = -CLOUD_DESCENT_V * (t - t_det)
    return C0 + np.array([0.0, 0.0, dz])

def m1_pos(t: float) -> np.ndarray:
    """导弹直线匀速指向原点"""
    v_dir = unit(ORIGIN - M1_0)
    v = MISSILE_SPEED * v_dir
    return M1_0 + v * t

def ray_sphere_intersection(ray_origin: np.ndarray, ray_dir: np.ndarray,
ray,
```

```

        sphere_center: np.ndarray, sphere_radius:
float) -> bool:
    """判断射线是否与球体相交"""
    oc = ray_origin - sphere_center
    a = np.dot(ray_dir, ray_dir)
    b = 2.0 * np.dot(oc, ray_dir)
    c = np.dot(oc, oc) - sphere_radius * sphere_radius
    discriminant = b * b - 4 * a * c

    if discriminant < 0:
        return False

    # 检查交点是否在射线正方向
    t1 = (-b - math.sqrt(discriminant)) / (2 * a)
    t2 = (-b + math.sqrt(discriminant)) / (2 * a)

    return t1 > 1e-6 or t2 > 1e-6 # 避免数值误差

def cylinder_silhouette_points(observer: np.ndarray, n_samples: int
= 36) -> List[np.ndarray]:
    """
    采样从观察者位置看圆柱体的轮廓关键点
    包括: 顶圆周边缘点、底圆周边缘点、以及侧面轮廓线上的点
    """
    points = []

    # 底面圆周点
    for i in range(n_samples):
        angle = 2 * math.pi * i / n_samples
        x = TRUE_TARGET_CENTER[0] + CYLINDER_RADIUS * math.cos(angl
e)
        y = TRUE_TARGET_CENTER[1] + CYLINDER_RADIUS * math.sin(angl
e)
        points.append(np.array([x, y, TRUE_TARGET_CENTER[2]]))

    # 顶面圆周点
    for i in range(n_samples):
        angle = 2 * math.pi * i / n_samples
        x = TRUE_TARGET_CENTER[0] + CYLINDER_RADIUS * math.cos(angl
e)
        y = TRUE_TARGET_CENTER[1] + CYLINDER_RADIUS * math.sin(angl
e)
        points.append(np.array([x, y, TRUE_TARGET_CENTER[2] + CYLIND
ER_HEIGHT]))

```

```

# 侧面轮廓线: 从观察者到圆柱轴的切线
obs_xy = np.array([observer[0], observer[1]])
center_xy = np.array([TRUE_TARGET_CENTER[0], TRUE_TARGET_CENTER
[1]])
to_obs = obs_xy - center_xy
dist_xy = np.linalg.norm(to_obs)

if dist_xy > CYLINDER_RADIUS: # 观察者在圆柱外部
    # 计算切点
    sin_alpha = CYLINDER_RADIUS / dist_xy
    cos_alpha = math.sqrt(1 - sin_alpha * sin_alpha)

    # 两个切点方向
    perp = np.array([-to_obs[1], to_obs[0]]) / dist_xy # 垂直向
量

    for sign in [-1, 1]:
        tangent_dir = cos_alpha * (to_obs / dist_xy) + sign * si
n_alpha * perp
        tangent_point_xy = center_xy + CYLINDER_RADIUS * tangent
_dir

        # 在切线上取多个高度的点
        for z in np.linspace(TRUE_TARGET_CENTER[2], TRUE_TARGET_
CENTER[2] + CYLINDER_HEIGHT, 5):
            points.append(np.array([tangent_point_xy[0], tangent
_point_xy[1], z]))

    return points

def is_completely_shielded_test(observer: np.ndarray, cloud_center_p
os: np.ndarray) -> bool:
    """
    判断云团球体是否完全遮蔽圆柱体
    """
    silhouette_points = cylinder_silhouette_points(observer, n_sampl
es=72)

    for target_point in silhouette_points:
        ray_dir = unit(target_point - observer)
        if not ray_sphere_intersection(observer, ray_dir, cloud_cent
er_pos, CLOUD_RADIUS):
            return False # 存在未被遮挡的射线

```



```

    return True

def find_complete_shielding_intervals_test(t0: float, t1: float, dt:
float = 0.01) -> List[Tuple[float, float]]:
    """寻找完全遮蔽的时间区间"""
    times = np.arange(t0, t1 + 1e-9, dt)
    shielded = []

    for t in times:
        observer = m1_pos(t)
        cloud_pos = cloud_center_test(t)
        shielded.append(is_completely_shielded_test(observer, cloud_
pos))

    # 提取连续的 True 区间
    intervals = []
    in_interval = False
    start_time = t0

    for i, (t, is_shielded) in enumerate(zip(times, shielded)):
        if is_shielded and not in_interval:
            in_interval = True
            start_time = t
        elif not is_shielded and in_interval:
            in_interval = False
            intervals.append((start_time, t))

    if in_interval:
        intervals.append((start_time, t1))

    refined_intervals = []
    for start, end in intervals:
        # 二分法精细化起始边界
        left, right = max(t0, start - dt), start + dt
        for _ in range(20):
            mid = 0.5 * (left + right)
            if is_completely_shielded_test(m1_pos(mid), cloud_center
_test(mid)):
                right = mid
            else:
                left = mid
        refined_start = right

```

```

# 二分法精细化结束边界
left, right = end - dt, min(t1, end + dt)
for _ in range(20):
    mid = 0.5 * (left + right)
    if is_completely_shielded_test(m1_pos(mid), cloud_center
_test(mid)):
        left = mid
    else:
        right = mid
refined_end = left

if refined_end > refined_start:
    refined_intervals.append((refined_start, refined_end))

return refined_intervals

def test_specific_parameters():
    """测试指定参数的遮蔽效果"""
    print('=== 测试指定参数的遮蔽时间 ===')
    print(f'测试参数: ')
    print(f' 飞行方向:  $\theta$  = {math.degrees(TEST_THETA):.2f}^\circ')
```

print(f' 飞行速度: v = {TEST_SPEED:.1f} m/s')

print(f' 投放时间: t_{drop} = {TEST_T_DROP:.2f} s')

print(f' 起爆延迟: τ = {TEST_TAU:.3f} s')

```

# 计算关键时刻和位置
t_det, C0 = detonation_state_test()
drop_point = uav_pos_test(TEST_T_DROP)

print(f'\n 关键信息: ')
print(f' 投放点: ({drop_point[0]:.1f}, {drop_point[1]:.1f}, {dro
p_point[2]:.1f})')
print(f' 起爆时刻:  $t$  = {t_det:.3f} s')
print(f' 起爆点: ({C0[0]:.1f}, {C0[1]:.1f}, {C0[2]:.1f})')
print(f' 起爆高度: {C0[2]:.1f} m')

# 检查参数合理性
if C0[2] <= 0:
    print('起爆点高度  $\leq 0$ , 烟幕弹已落地! ')
    return

# 计算遮蔽时间
t0 = t_det
t1 = t_det + CLOUD_EFFECT_TIME

```

```

print(f'\n 计算遮蔽时间 (时间窗口: {t0:.3f}s - {t1:.3f}s) ...')
intervals = find_complete_shielding_intervals_test(t0, t1, dt=0.
02)
total_time = sum(b - a for a, b in intervals)

print('\n=== 遮蔽时间结果 ===')
if intervals:
    for i, (a, b) in enumerate(intervals, 1):
        print(f'遮蔽区间{i}: [{a:.3f}, {b:.3f}] s (长度 {b - a:.
3f} s)')
    else:
        print('无完全遮蔽时段')

print(f'\n 总遮蔽时长: {total_time:.3f} s')

# 额外验证: 检查几个关键时刻的遮蔽状态
print(f'\n=== 关键时刻验证 ===')
test_times = [t_det, t_det + 1, t_det + 5, t_det + 10, t_det + 1
5]
for t_test in test_times:
    if t_test <= t1:
        observer = m1_pos(t_test)
        cloud_pos = cloud_center_test(t_test)
        is_shielded = is_completely_shielded_test(observer, clou
d_pos)
        print(f't = {t_test:.1f}s: {"完全遮蔽" if is_shielded else "未完全遮蔽"}')

if __name__ == '__main__':
    test_specific_parameters()

```